LOOPI

Learn. Review. Repeat.

ASSIGNMENT 2

Build Your Proposed Educational App

Module: IBM4203 – Mobile Application Development Using Android
Session: August 2026
Programme: BCSI / BITI
Lecturer: Lai Mei Yoon
Coursework: Assignment 2 (Group of 5) – 30%
Group: Group 6

No.	Name	Student ID
M1	Akhmetova Azhar	I26043015
M2	Tussupova Dariya	I26043025
M3	Zhassyuzak Nursaule	I26043027
M4	Suzain Mudabbir Sayyad	I26043300
M5	Sanjeet Sachin Zele	I26043274

Submission date: _____

Assessment Rubric

Criteria	Excellent	Good	Satisfactory	Needs improvement	Score
Functionality (Core features work)	All required features implemented and work flawlessly. All screens functional (13–16)	Most features work with minor issues (9–12)	Some features work; significant issues present (5–8)	Few features work; major issues (1–4)	/16
UI/UX Design (Compose & Material 3)	Intuitive, polished interface using Material 3 components and Compose best practices (13–16)	Good interface with minor usability issues (9–12)	Basic interface with several usability issues (5–8)	Poor interface with major usability issues (1–4)	/16
Data Handling (Room Database)	Efficient Room CRUD with proper DAO, entities, and ViewModels. Data persists correctly (10–12)	Room CRUD works with minor issues (7–9)	Partial Room implementation with significant issues (4–6)	Poor or missing Room implementation (1–3)	/12
Navigation & Architecture (NavController + ViewModel)	Smooth multi-screen navigation; ViewModel separates UI and data logic cleanly (7–8)	Functional navigation with minor architectural issues (5–6)	Limited navigation or mixed concerns (3–4)	No navigation or ViewModel (1–2)	/8
Collaboration & Task Division	Clear, balanced task division documented; all members contributed meaningfully (7–8)	Reasonable division with some imbalance (5–6)	Uneven contribution with limited documentation (3–4)	No evidence of teamwork or task division (1–2)	/8
Code Quality	Clean, readable, well-organised Kotlin code following Compose conventions (7–8)	Mostly clean code with minor deviations (5–6)	Somewhat organised with several issues (3–4)	Poorly organised, difficult to read (1–2)	/8
Documentation & Report	Complete, well-written report with all required sections and task contribution table (10–12)	Clear report with minor details missing (7–9)	Basic report with some information missing (4–6)	Incomplete or poorly organised report (1–3)	/12
Personal Reflection (Individual)	Deep, authentic reflection on learning process, user empathy, and growth as a future developer (7–8)	Thoughtful but surface-level reflection (5–6)	Generic statements with limited personal voice (3–4)	Absent or AI-generated sounding reflection (1–2)	/8
Presentation & Demo	Engaging, structured demo. Effectively explains architecture and features (10–12)	Clear demo and structured explanation (7–9)	Basic demo with limited explanation (4–6)	Incomplete demo or fails to communicate design (1–3)	/12
TOTAL					/100

Lecturer Marking and Comments

This page is left for the lecturer to insert the marking table and comments.

Criteria (max score)	Score
Functionality (16)	
UI/UX Design (16)	
Data Handling (12)	
Navigation & Architecture (8)	
Collaboration & Task Division (8)	
Code Quality (8)	
Documentation & Report (12)	
TOTAL (80)	
Penalty (Plagiarism / Late Submission / High Similarity or AI Index)	
ADJUSTED GROUP TOTAL	

Individual Mark	M1	M2	M3	M4	M5
Personal Reflection (8)					
Presentation Delivery (12)					
Penalty (Minimal Contribution or Poor Peer Evaluation)					
Total (100)					
Scaled to 30%					

Comments:

Table of Contents

(If page numbers are missing, right-click the table in Word and choose Update Field.)

1. App Title, Objectives and Target User

1.1 App title

Our app is called **Loopi – Learn. Review. Repeat.** It is an offline-first Android study and revision app that we built in Kotlin with Jetpack Compose, Material 3 and Room. In Assignment 1 we proposed it as "Loopi (Learn. Practice. Remember)". During development we changed the tagline to "Learn. Review. Repeat." because it describes the loop the learner actually goes through in the app: study a topic, review it with confidence checks, and repeat the weak parts. The product idea itself did not change.

1.2 Problem we are solving

In Assignment 1 we identified three problems from our research and our survey of 35 learners, most of them university students aged 18 to 21 from Kazakhstan and India (Group 6, 2026, pp. 16–21, 28):

- Students understand a topic in class but forget it before the exam.
- Students feel confident about a subject without realising they still make mistakes.
- Study materials are spread across different places, which makes revision difficult.

Our survey also showed that students were frustrated by too many advertisements and expensive subscriptions (54.3%), wanted progress tracking (20%) and interactive quizzes (14.3%), and wanted less dependence on the internet. The most common rating for existing apps was only 3 out of 5 (57.1%).

1.3 Objectives

1. Let university students create and organise their own study topics, flashcards and learning resources in one place.
2. Support active recall through flashcards and multiple-choice quizzes that the learner writes, or that Loopi suggests from the learner's own PDF notes.
3. Ask the learner how confident they are **before** the answer is revealed, then compare that confidence with the actual result and give a short explanation.
4. Show progress by topic, highlight weak spots and give a confidence calibration score so students can see where they are overconfident.
5. Keep all study data on the device so every core feature works without an internet connection, an account server or advertisements.
6. Allow students to share study packs with classmates as a file, without an online account.

1.4 Target user

Our target user, from Assignment 1, is a **university student who wants to revise and consolidate their learning** and wants a simple way to file topics and pick out the ones that need more attention. A typical persona is a second-year IT student preparing for exams in Databases, Networks, Algorithms and Operating Systems, who studies on the bus or in places with poor Wi-Fi, has lecture slides as PDFs, and has limited time each day. This persona shaped our visual design: a calm blue and white theme, large readable cards, a daily goal of 20 cards, a short session length, and an app that never asks for an internet connection.

1.5 What a learner can do with Loopi

- Create a local account and sign in, with the data staying on the phone.
- Create topics with a title, description, category, difficulty and target number of cards, with validation.
- Add flashcards and resources (notes, links and PDF files) to each topic.
- Turn a PDF of lecture notes into reviewable multiple-choice questions, or into a complete new topic, entirely offline.
- Study with flip cards, or take a one-screen quiz with a confidence rating before the answer is revealed.
- See accuracy, weak spots, streaks and a confidence calibration score in Progress.
- Create, export and import study packs, join local study groups and report a problem.

2. Assignment 1 App Proposal Blueprint

Our Assignment 1 proposal blueprint is reproduced below as the specification for this assignment. It describes Loopi as a study app that helps university students practise, review topics and track their progress, with confidence ratings before answers are shown, progress saved on the device, and sharing of flashcard sets as a possible extra feature.



Figure 1. Loopi App Proposal Blueprint, reproduced from our Assignment 1 report, p. 28 (Group 6, 2026).

Before coding, we turned the blueprint into a wireframe and feature breakdown (Figure 2). It divides the app into core flows (Flows 2 to 6), supporting flows (Flow 1 and Profile/Settings) and future extensions (Flows 7 and 8). We used this sheet to divide the work between the five of us.

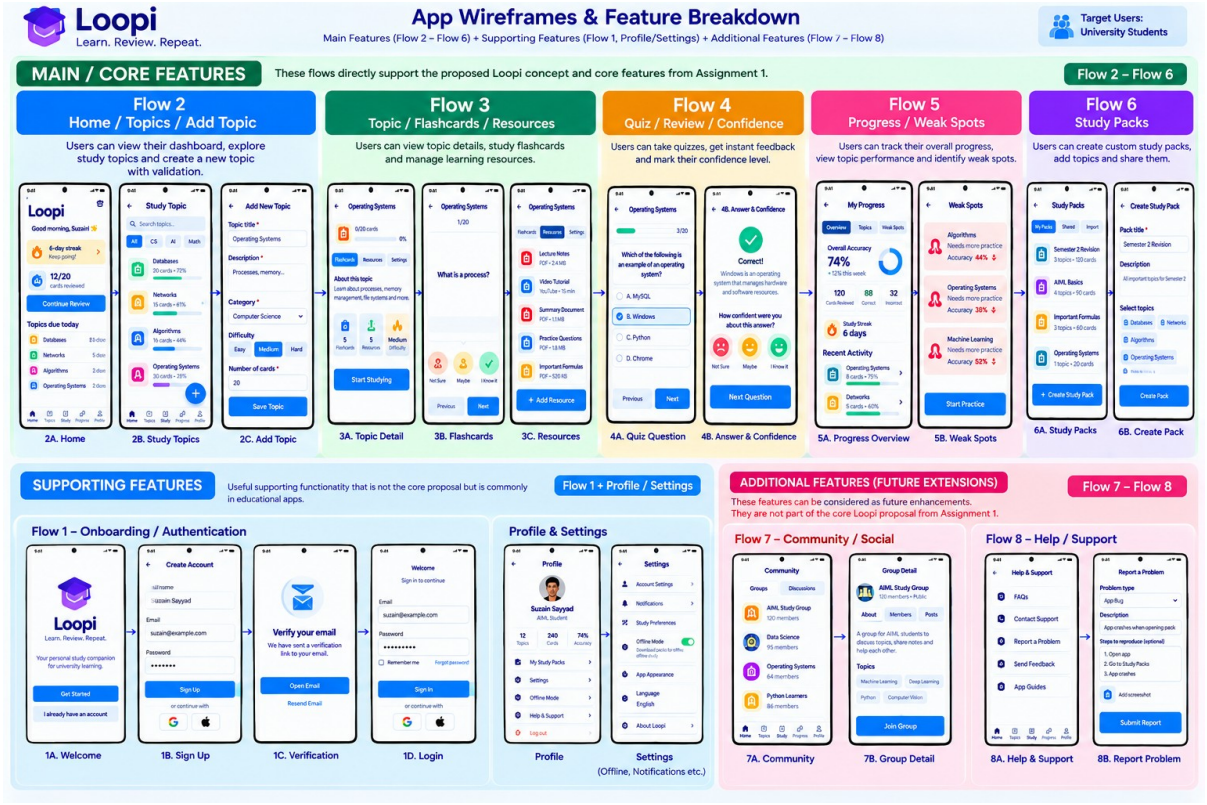


Figure 2. Loopi App Wireframes & Feature Breakdown that we used to plan Assignment 2.

2.1 How the final app compares with the proposal

Proposed in Assignment 1	What we built	Status
Create own flashcards/tests for a subject	Topics with flashcards, editable questions, answers, explanations and wrong options; quizzes built from the cards	Done
Rate confidence before seeing the answer	One-screen quiz: choose answer and Not sure / Maybe / I know it, then Check answer reveals the result	Done
Compare confidence with the result, read a short explanation, try again	Inline feedback, an "Explain simply" box, a personal calibration message, and a practice round for weak cards	Done
Show which topics need more attention	Weak-spot card with a Practice button, topic accuracy, topics due today on Home	Done
Show progress by topic	Progress tab with accuracy chart (week/month/all time), correct/incorrect, answer time and calibration score	Done
Save learning materials on the device	Resources per topic: notes, links and PDF files stored in Room	Done
Work without an internet connection	Everything is stored in Room and SharedPreferences; PDF reading and question generation run on the phone	Done
Share flashcard sets with classmates (extra)	Study packs exported and imported as .json files that carry topics and flashcards	Done
Remind students when it is time to review	Daily goal and reminder preferences are saved; scheduled push notifications are not yet sent	Partial
Schedule topics for review over	Topics due today and weak-card practice are based on	Partial

Proposed in Assignment 1	What we built	Status
time	accuracy; a full spaced-repetition schedule is future work	
Not in proposal: PDF to MCQ	Offline generator that suggests questions from a PDF, with a review step before saving; it can add cards to a topic or create a whole new topic	Added
Wireframe Flows 7–8 (future extensions)	Local Community groups and posts; Help & Support with FAQs, guides and a Report a Problem form	Added (local only)

Overall, the app we built follows our proposal closely. The main differences are that reminders are saved as a setting rather than sent as notifications, and that we added the PDF-to-quiz feature, the confidence calibration score and the local Community and Help screens, which were not part of the original core proposal.

3. Features and Functions

This section walks through every main screen of the app, flow by flow, using screenshots that we captured on the Android emulator from the final build. All screens share one Loopi theme (LoopiColors and LoopiTheme): a blue gradient header that runs under the status bar, white cards with soft shadows, coloured icon tiles for each topic, and a five-tab bottom navigation bar (Home, Topics, Study, Progress, Profile).

3.1 Flow map

Flow	Screens	Main source files
1. Onboarding & sign-in	Splash, Welcome, Sign Up, Sign In, Forgot/Reset Password, First-time setup	flow1screens.kt, MainActivity.kt
2. Home, Topics, Add Topic	Home dashboard, Study Topics with search and filter, Add/Edit Topic	flow2screens.kt, TopicEntity.kt, TopicDao.kt
3. Topic, Flashcards, Resources	Topic detail, flashcard list and editor, flip-card study, resources list and editor	Flow3Screens.kt, Flow3StudyScreen.kt, FlashcardEntity.kt, ResourceEntity.kt
3+. PDF to quiz	Quiz from PDF for an existing topic; New Topic from PDF from Add Topic; both read the PDF offline and let the learner review questions before saving	PdfQuizScreen.kt, McqGenerator.kt
4. Quiz & confidence	Review settings, one-screen question with confidence, session results, weak-card practice	Flow4Screens.kt, ReviewResultEntity.kt, ReviewResultDao.kt
5. Progress & weak spots	Your Progress, topic progress, Profile, Settings	Flow5Screens.kt
6. Study packs	My packs, Create pack with topic checklist, pack detail with export/share, import	Flow6Screens.kt, StudyPackEntity.kt, StudyPackDao.kt
7–8. Community & Help	Community groups and discussions, group detail, Help & Support, Report a Problem	Flow78Screens.kt

3.2 Flow 1 – Onboarding and local sign-in

The Welcome screen introduces Loopi with four short promises: study smarter, find weak topics, track your progress and works offline. A learner creates a local account with their full name, university email and a password of at least 6 characters with a number. The sign-up form validates every field and shows an error under the field that needs fixing. Sign In has an eye toggle to show the password and a Forgot password link that uses a recovery code. A green note tells the learner that their account and study data stay on the phone. Once signed in, Loopi remembers the user and opens straight on Home the next time.

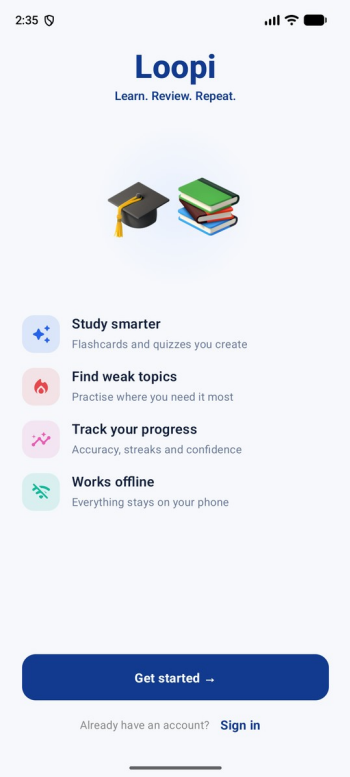


Figure 3. Welcome screen

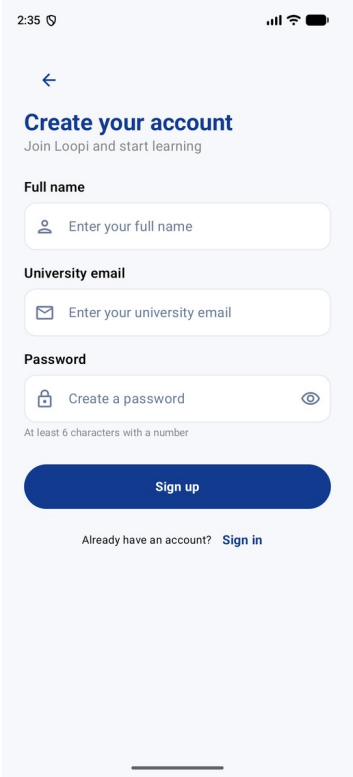


Figure 4. Sign Up with validation

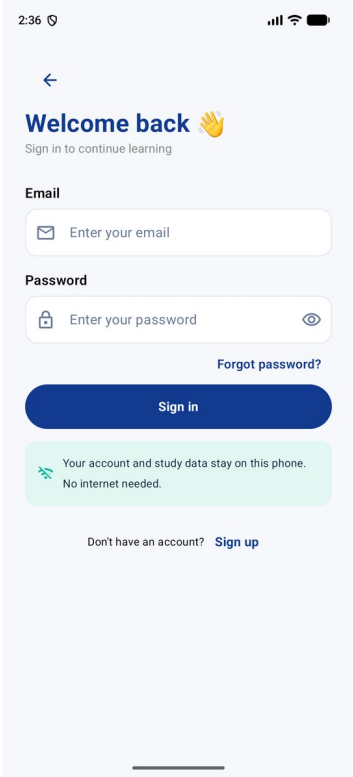


Figure 5. Sign In (local account)

Passwords are never stored as plain text. We hash them with PBKDF2 and a random salt and keep only the hash in SharedPreferences (a unit test, PasswordHashTest, checks this). Because there is no server, Google/Apple sign-in and email verification from the wireframe were left out on purpose; they were never part of our Assignment 1 proposal.

3.3 Flow 2 – Home dashboard, Study Topics and Add Topic

Home greets the learner by time of day and shows a daily-goal ring (for example 2/20 cards, 10%), the day streak, the number of topics due and the number of cards seen. Coloured quick actions open Study, Add topic, Packs and Groups, and a **Continue review** button jumps into the next session. The **Topics due today** list shows topics that still need practice.

Study Topics is a LazyColumn of topic cards. Each card shows a coloured icon tile, the title, the number of cards against the goal, the accuracy percentage and a progress bar, and opens the topic detail when tapped. There is a search bar, a Filter / Sort screen (by category, difficulty and name) and a floating + button to add a topic.

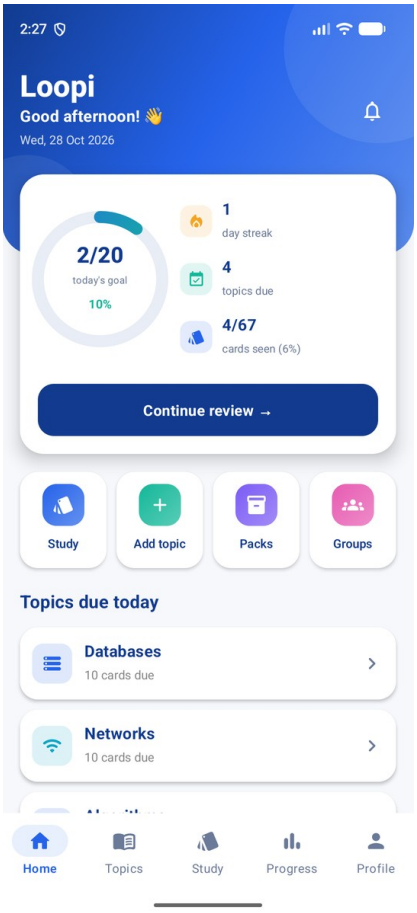


Figure 6. Home dashboard

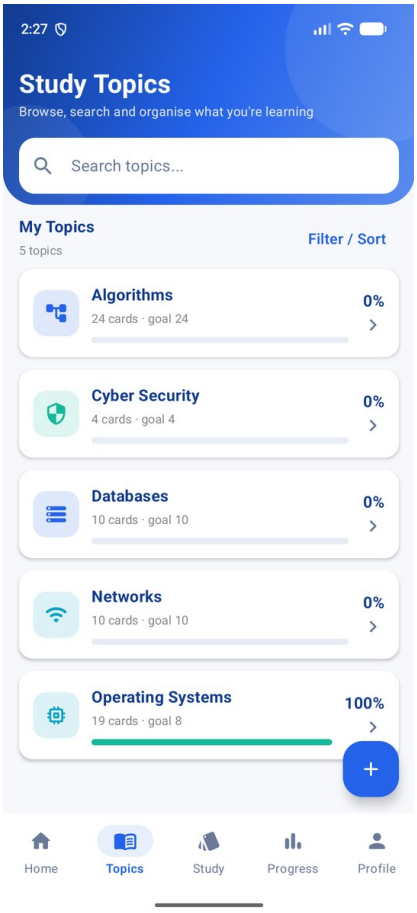


Figure 7. Study Topics list

Add Topic asks for a title, description, category (DropdownMenu), difficulty (Easy, Medium or Hard) and the number of cards. Every required field is validated when the learner presses Save: "Title is required", "Description is required", "Please select a category" and "Number of cards is required" appear under the fields in red, and a Snackbar confirms when a topic is saved, updated or deleted. Topics can be edited and deleted from the topic screen.

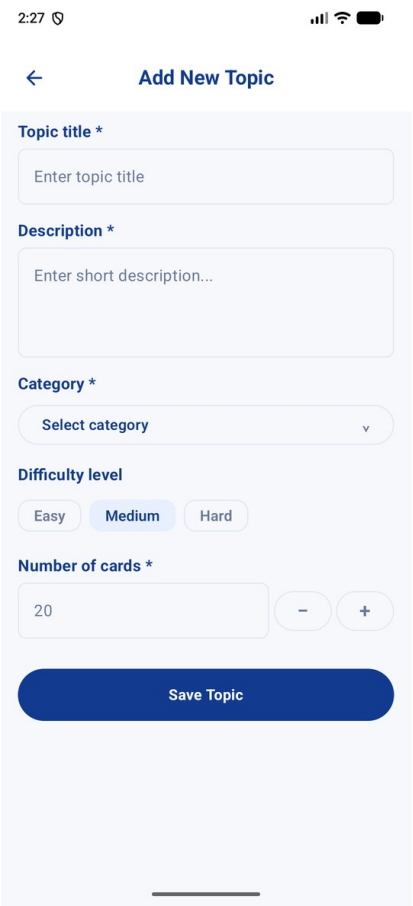


Figure 8. Add Topic form

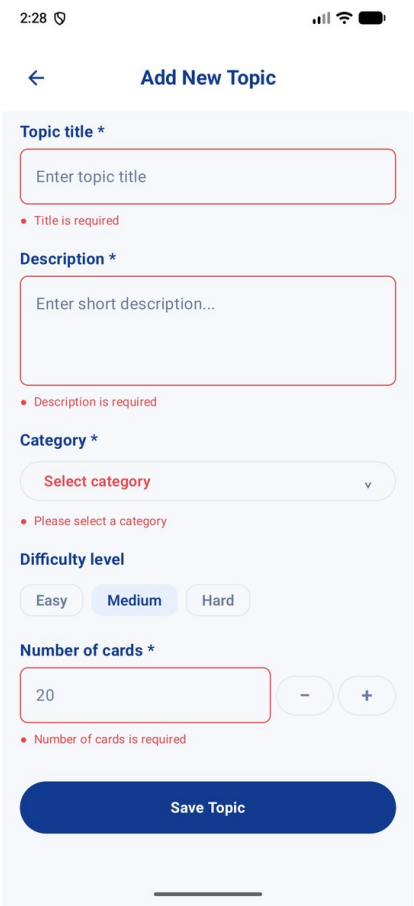


Figure 9. Validation errors on Add Topic

Creating a whole topic from a PDF

Add Topic also starts with a **Create from a PDF instead** card. The learner picks a PDF and Loopi reads it on the phone, then fills in the topic for them: the title comes from the first heading in the document (or the file name), the description from its first full sentence, and the category from key words in the notes (for example "network" and "protocol" suggest Information Technology). On the **New Topic from PDF** screen the learner can change the title, description and category and keep, edit or untick each generated question. Then they tap **Create topic with 7 cards**. Loopi saves the topic with its cards, adds the PDF to the topic's Resources and opens the new topic. When we tested it with a Software Engineering lecture PDF, it created a "Software Engineering" topic with 7 cards.

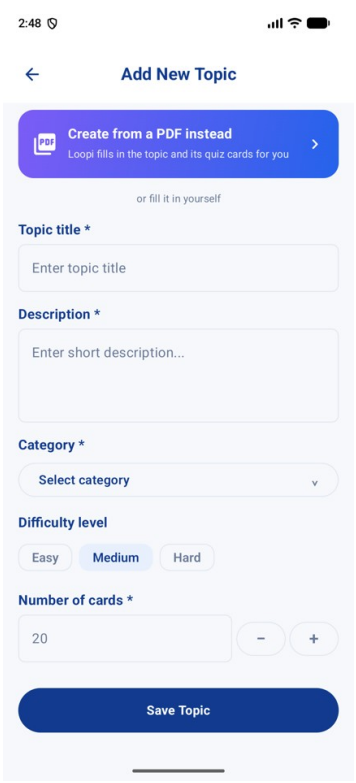


Figure 10. Add Topic: Create from a PDF instead

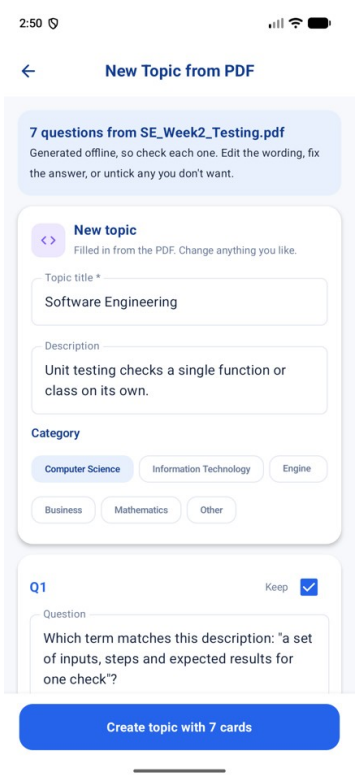


Figure 11. New Topic from PDF: review title, category and cards

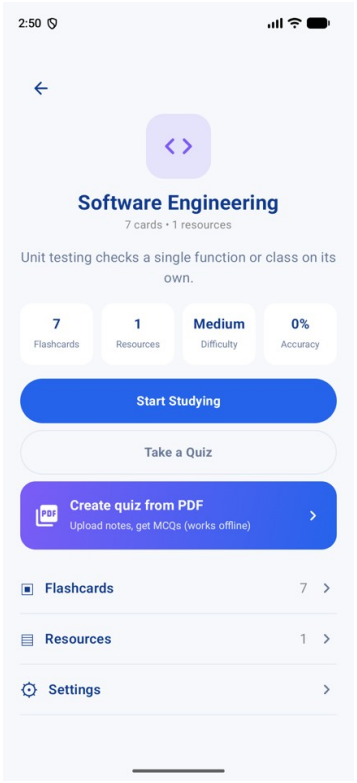


Figure 12. The new topic with 7 cards and the PDF in Resources

3.4 Flow 3 – Topic detail, flashcards, flip-card study and resources

The **topic detail** page is the hub for one topic. It shows the topic's icon, description and four stat tiles (flashcards, resources, difficulty and accuracy), and the actions **Start Studying**, **Take a Quiz** and **Create quiz from PDF**, followed by links to Flashcards, Resources and topic Settings.

The topic screens use Flashcards, Resources and Settings tabs, like the wireframe. The **Flashcards** list has a search bar and shows every card in the topic and lets the learner add, edit and delete cards. A card has a question, an answer, an optional explanation and, for PDF-generated cards, its own wrong options. **Start Studying** opens the flip-card mode: the learner sees one side, taps to flip it, and rates it with Not sure, Maybe or I know it, moving with Previous and Next. **Resources** keep notes, web links and PDF files for the topic, each with a title, description and tags.

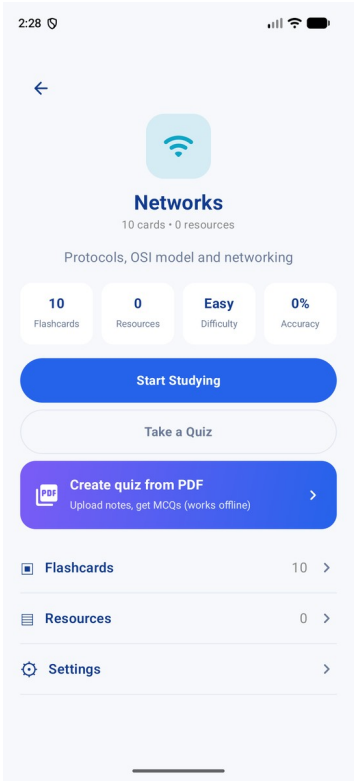


Figure 13. Topic detail

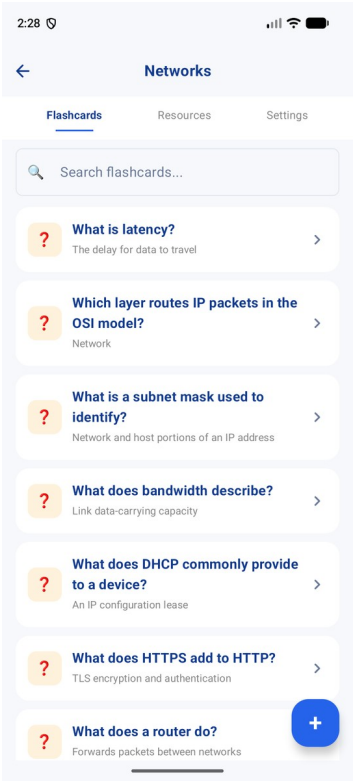


Figure 14. Flashcards in a topic

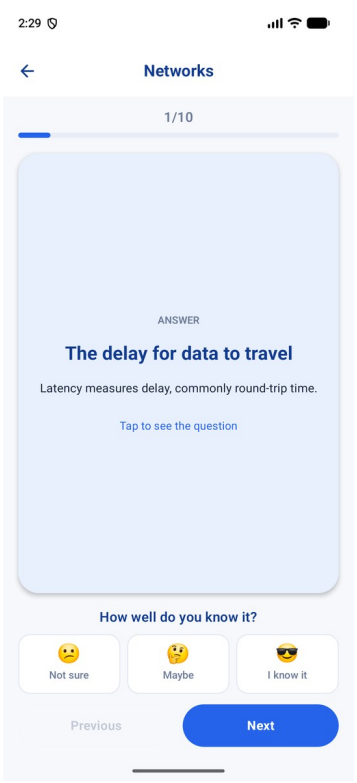


Figure 15. Flip-card study mode

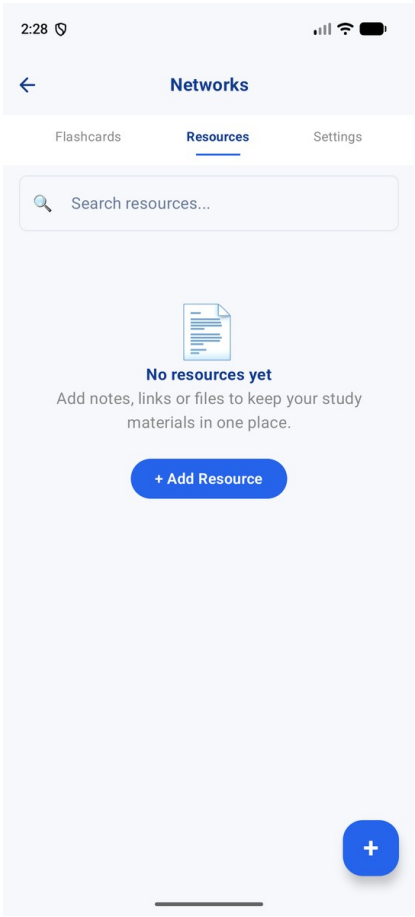


Figure 16. Resources for a topic

3.5 Creating quiz questions from a PDF (offline)

We added a feature that was not in our proposal but solves a real problem we saw: students already have their lecture slides and do not want to type every card. From a topic, the learner taps **Create quiz from PDF** and picks a PDF. Loopi reads the text on the phone with PdfBox-Android (up to 40 pages), and our own McqGenerator finds definitions such as "A process is a program in execution..." or "Firewall: a device that..." and key terms that repeat in the document. It turns them into "Which term matches this description..." and fill-in-the-blank questions, using other terms from the same PDF as wrong options.

Because the questions are generated by rules rather than AI, the learner reviews every question before it is saved. They can edit the wording, correct the answer or untick questions they do not want, then save them as flashcards. The PDF is also added to the topic's resources. The same generator powers **Create from a PDF instead** on Add Topic (Section 3.3). In our test with an Operating Systems lecture PDF it produced 11 questions, all of which were usable. It works best on notes that define terms; scanned PDFs have no text layer, and Loopi shows a clear message that the text cannot be read. An online AI service could write better questions, but we kept the feature fully offline to match our proposal.

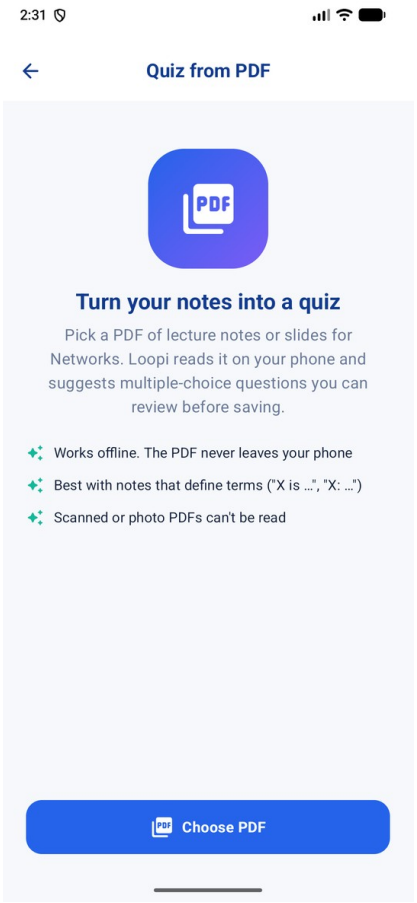


Figure 17. Quiz from PDF: choose a PDF

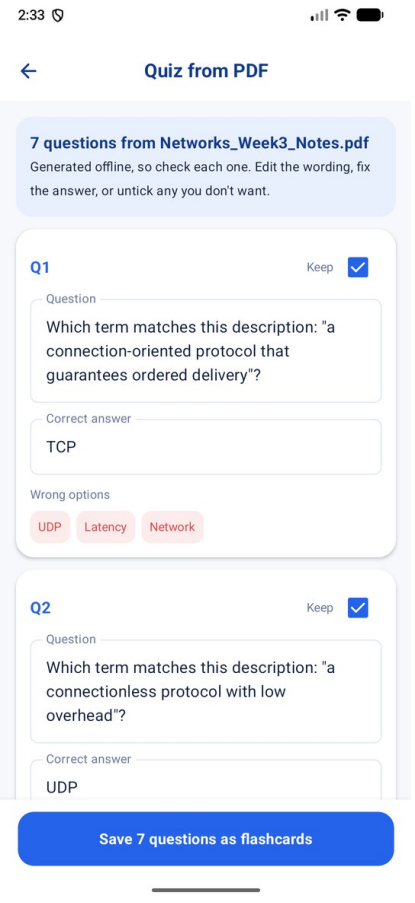


Figure 18. Review and edit generated questions

3.6 Flow 4 – Quiz with confidence before the answer

This is the core of Loopi. Each question is one screen with a progress bar (Question 1 of 10), four options and three confidence buttons. The bottom button guides the learner: it says "Choose an answer", then "Rate your confidence", and only when both are chosen does it become **Check answer**.

After checking, the correct option turns green, a wrong choice turns red, and a feedback card appears with a personal message that compares confidence with the result (for example "You felt sure, but this one needs another look. Added to your weak spots."), an **Explain simply** box and the correct answer. The button then becomes **Next question** or **See results**.

Every answer is saved in Room with the question, the learner's answer, the correct answer, whether it was correct, the confidence, the response time and the time of the review. At the end of a session the learner sees their score, and wrong or unsure cards can be practised again in a short weak-card round. Review settings let the learner choose the number of cards and whether to review weak cards only.

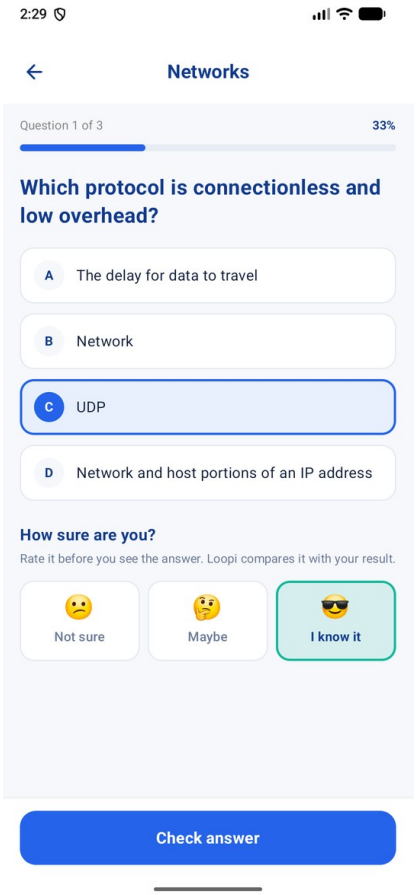


Figure 19. Answer and confidence chosen, before Check answer

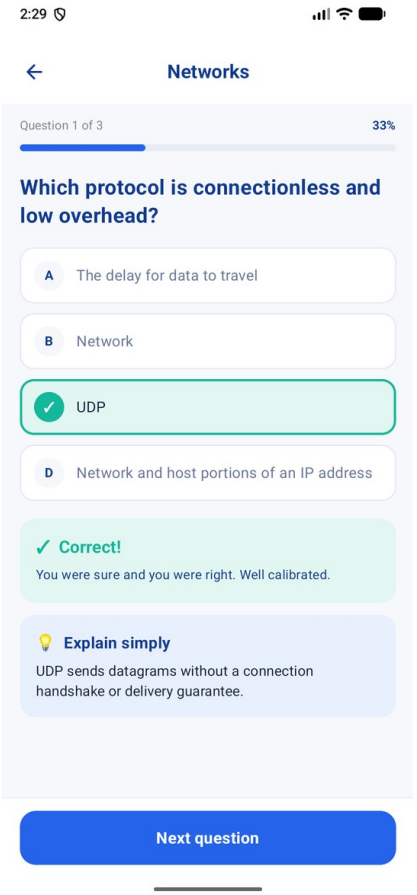


Figure 20. Feedback after Check answer

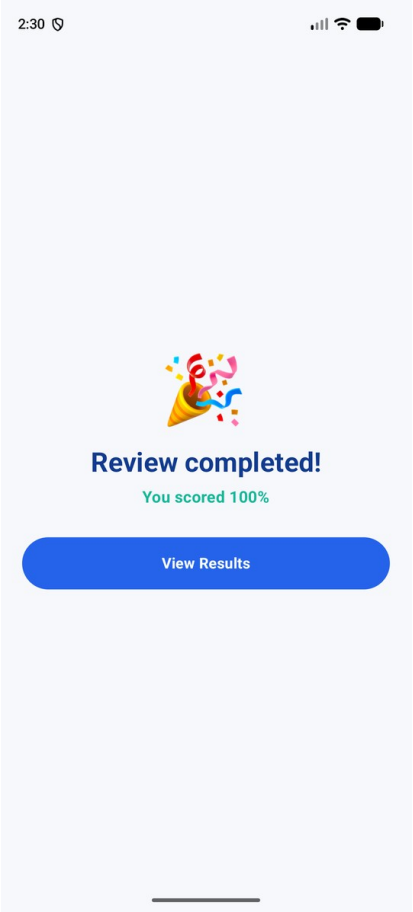


Figure 21. Review completed

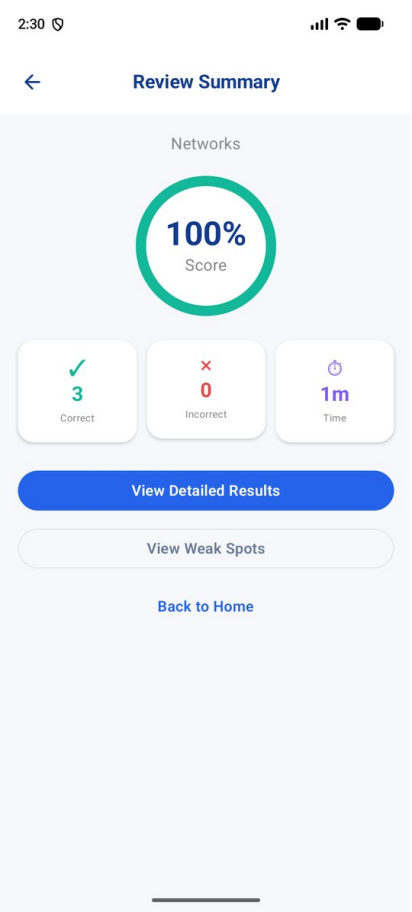


Figure 22. Review Summary with View Weak Spots

Our first version used four separate screens per question (answer, correct/incorrect, explanation, confidence) and asked for confidence after the answer was shown. We redesigned it into one screen so that confidence is given before the answer is revealed, as our proposal intended, and so that a ten-card session no longer takes forty screens.

3.7 Flow 5 – Progress, weak spots and confidence calibration

Your Progress shows accuracy per answer as a bar chart for the selected period (Week, Month or All time), with accuracy, correct and incorrect totals, average answer time and the number of topics practised. A red **Weak spot** card names the weakest topic (for example "Databases is at 0%") with a **Practice** button that starts a short round. The **Confidence check** card shows a calibration score, which measures how well the learner's confidence matched their results, and a hint such as "You were sure but wrong 1 time(s). Slow down on cards that feel easy." Tapping a topic opens its own progress page.

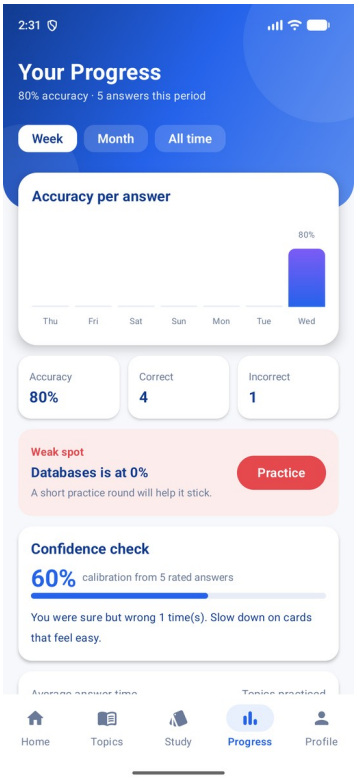


Figure 23. Your Progress with weak spot and calibration

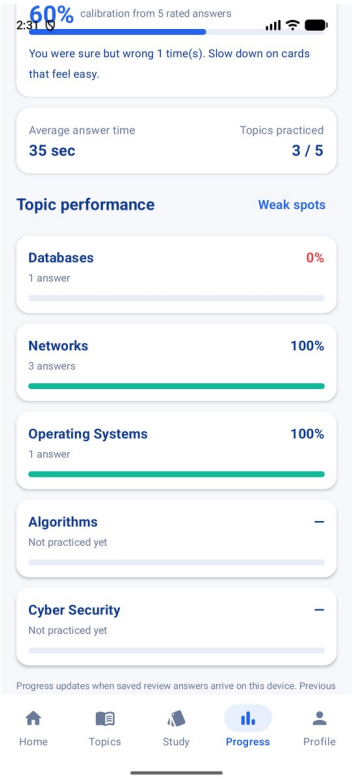


Figure 24. Topic performance on Progress

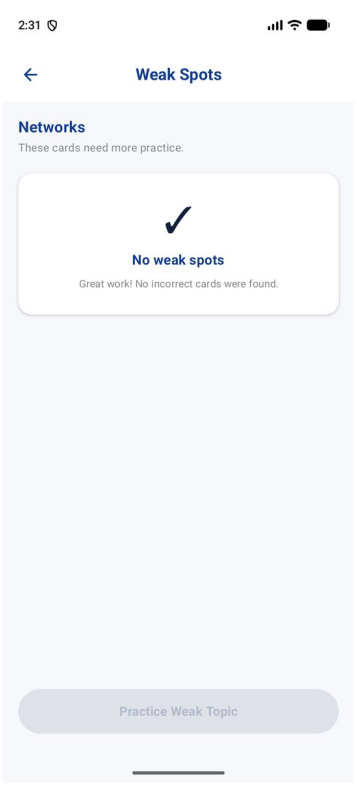


Figure 25. Weak Spots (empty after a perfect round)

3.8 Profile and Settings

Profile shows the learner's name and email, their streak, and totals for topics, cards and accuracy, with links to Edit Profile, My Study Packs, Community, Settings and Help & Support, and Log out. **Settings** holds the reminder switches (study reminders, review due, weak topic alerts), the daily card goal (10, 20, 50 or 100), the default review mode (all cards or weak cards only) and app information. The reminder switches are labelled in the app as saved on this device, because scheduling is not enabled yet. All settings are saved in SharedPreferences so they persist after the app is closed.

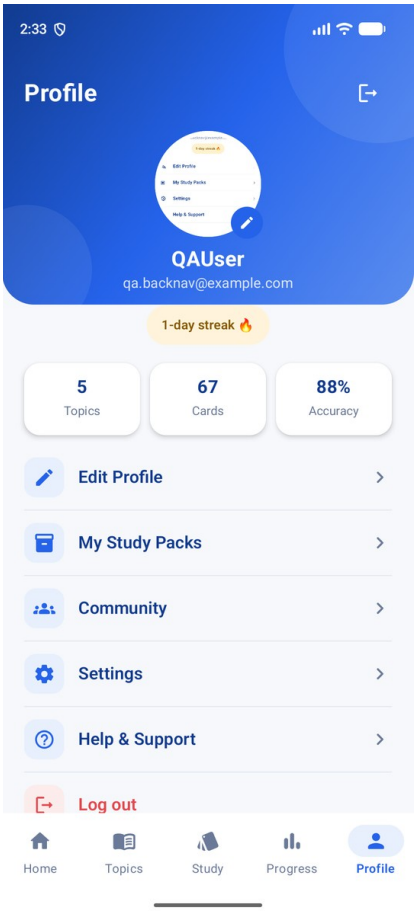


Figure 26. Profile

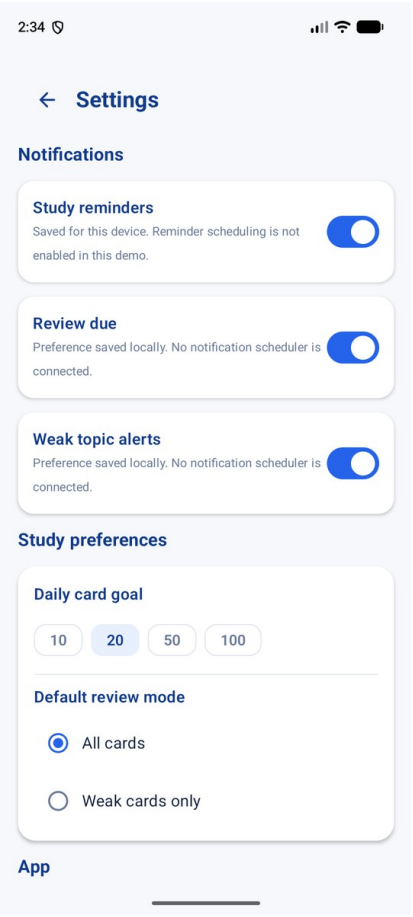


Figure 27. Settings

3.9 Flow 6 – Study packs with export and import

A **study pack** is a named group of topics, for example "Semester 2 Revision". On Create Pack the learner enters a title (required) and description and ticks topics from a checklist. On the Pack Preview screen they can **Export** the pack to a .json file or **Share** it with Android's share sheet. The exported file carries every topic's description, category, difficulty and flashcards, so a classmate can **Import pack** on their own phone and receive the topics and cards even if they never created them. The import checks the file and reports what was added, such as "Cyber Security Basics imported with 1 new topic and 4 cards".

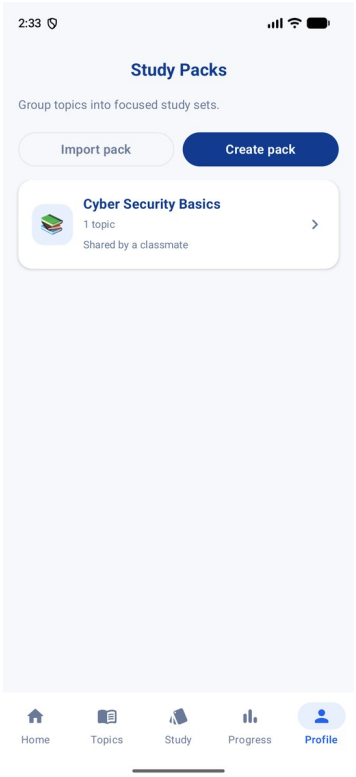


Figure 28. Study Packs

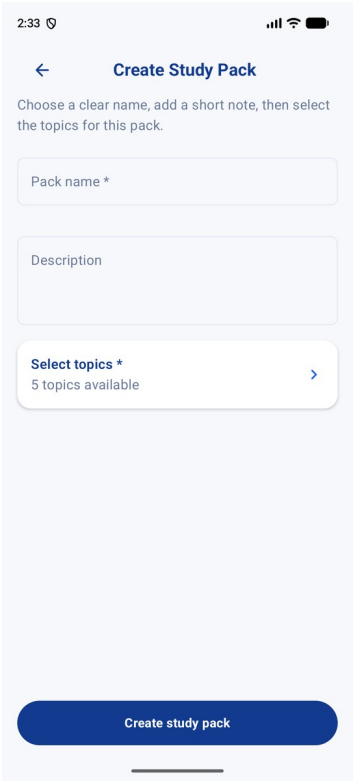


Figure 29. Create Pack with topic checklist

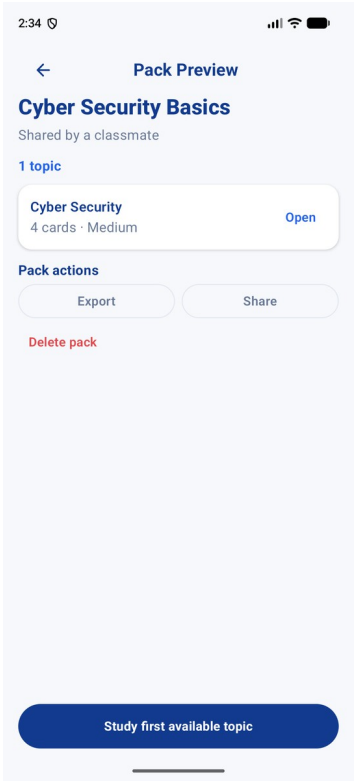


Figure 30. Pack detail with Export and Share

3.10 Flows 7 and 8 – Community and Help & Support

The wireframe listed Community and Help as future extensions. We implemented both as local features, clearly labelled in the app. **Community** has Groups and Discussions tabs. A learner can join or leave a study group such as "SQL Study Circle", open its About, Members and Posts tabs, and write their own posts. A note at the top says that groups and posts are saved on this device until Loopi has a sync server, which meets the brief's rule that placeholder social features must be labelled. **Help & Support** has frequently asked questions (where is my data saved, how does the confidence check work, what counts as a weak spot, how do I share a pack, can I use Loopi offline), short app guides, Contact us and a **Report a Problem** form with a problem type dropdown, a description, optional steps to reproduce and an optional screenshot, which is validated and saved on the device.

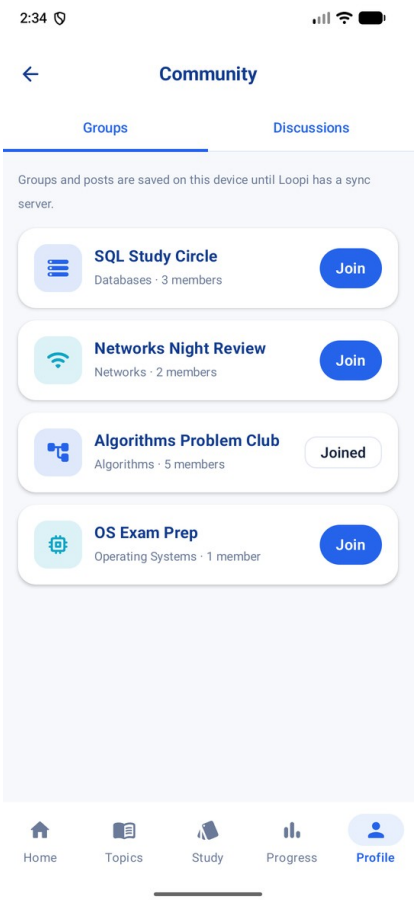


Figure 31. Community groups

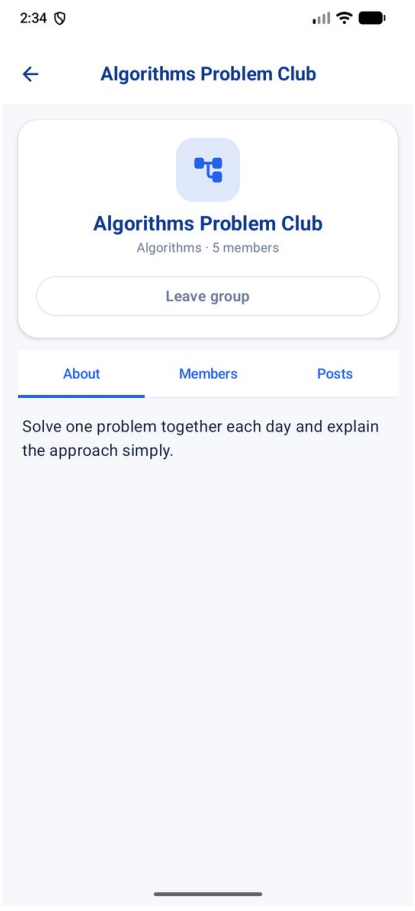


Figure 32. Group detail and posts

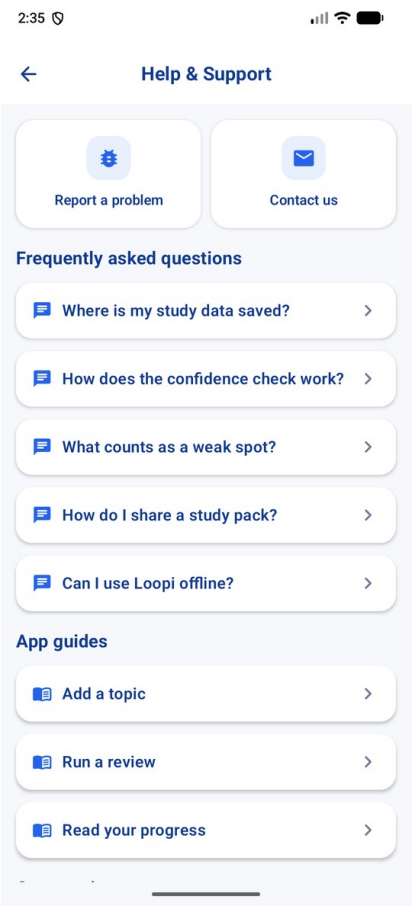


Figure 33. Help & Support

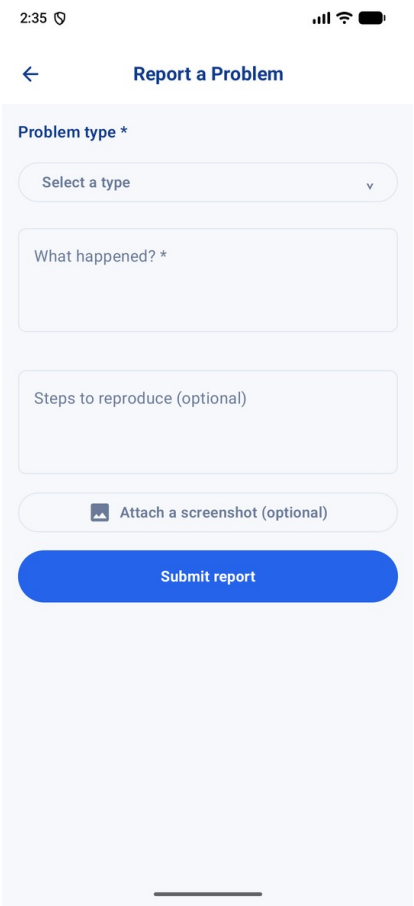


Figure 34. Report a Problem form

3.11 How the app meets the Assignment 2 requirements

Requirement in the brief	Where it is met in Loopi
1. Layout & UI: Column, Row, Box, Card, Modifier; Material 3 theme and typography; design for the persona	All screens; shared LoopiTheme, LoopiColors and Type.kt; LoopiHero gradient header and QuickActionTile in LoopiComponents.kt
2. State management: remember, mutableStateOf, state holders	Form fields and quiz selections use remember/mutableStateOf; LoopiAppState, Flow3AppState, Flow4AppState and Flow6AppState hold screen state and collect Room Flows
3. Scrollable list with text + another element, tappable to detail	Study Topics, Flashcards, Resources, Study Packs and Community are LazyColumns; topic items show an icon tile, percentage and progress bar and open Topic detail
4. Navigation: NavController, NavHost, ≥3 screens, arguments	One NavHost in MainActivity.kt with 50+ routes; arguments such as topic_detail/{topicId}, edit_topic/{topicId}, review_question/{topicId}, study_pack_preview/{packId}
5. Two or more input types, validation, Toast/Snackbar	TextField, DropdownMenu, RadioButton, Checkbox, Switch and option buttons; required-field errors on all forms; Snackbar and status messages after save, delete, export and import
6. Room entity, DAO with insert/update/delete/query, AppDatabase, persistence	Five entities, five DAOs, LoopiDatabase (version 8 with migrations); data stays after closing and reopening the app

Requirement in the brief	Where it is met in Loopi
7. Educational theme and labelled placeholders	Confidence-before-answer quiz, weak spots and calibration from our proposal; Community clearly labelled as saved on this device only

4. Database Design – Room

We use Room as the local SQLite layer. The database file is called loopi_database and is declared in LoopiDatabase.kt with five entities. The schema is now at **version 8**; seven migrations (1→2 up to 7→8) add columns and tables step by step, so a learner who installed an older build keeps their data after updating. Sign-in details, settings, community posts and problem reports are small key-value data, so we keep them in SharedPreferences rather than Room.

4.1 Entities

Table / Entity	Fields	Purpose
topics / TopicEntity	id (PK, auto), title, description, category, difficulty, numberOfCards, targetCardCount, progress, lastUpdated	A study topic and its summary: goal and current accuracy.
flashcards / FlashcardEntity	id (PK), topicId (indexed), question, answer, explanation, wrongOptions, createdAt, updatedAt	A question/answer card. wrongOptions keeps the distractors of PDF-generated questions.
resources / ResourceEntity	id (PK), topicId (indexed), type, title, content, url, fileUri, description, tags, createdAt, updatedAt	Notes, links and PDF files attached to a topic.
review_results / ReviewResultEntity	id (PK), topicId, flashcardId (both indexed), question, correctAnswer, explanation, userAnswer, isCorrect, confidence, responseTimeMillis, reviewedAt	One row per answered question. It powers feedback, progress, weak spots and calibration.
study_packs / StudyPackEntity	id (PK), name (indexed), description, topicNamesJson, createdAt, updatedAt	A named collection of topics that can be exported and imported.

4.2 Relationships

One topic has many flashcards, many resources and many review results (one-to-many through topicId). A review result also points to the flashcard that was answered (flashcardId). A study pack refers to its topics by title, stored as a JSON array, because titles still make sense when the pack is imported on another phone where the topic ids are different.

Relationship	Type	How it is kept consistent
TopicEntity → FlashcardEntity	1 : N via topicId	deleteFlashcardsForTopic() is called when a topic is deleted
TopicEntity → ResourceEntity	1 : N via topicId	deleteResourcesForTopic() is called when a topic is deleted
TopicEntity → ReviewResultEntity	1 : N via topicId	clearTopicResults() resets a topic's history
FlashcardEntity → ReviewResultEntity	1 : N via flashcardId	Results keep a copy of the question text, so history still reads correctly after a card is edited
StudyPackEntity → TopicEntity	N : M by title (JSON)	Topics are looked up with getTopicByTitle() on export and import

We indexed every topicId and flashcardId column for fast lookups. We did not declare Room @ForeignKey constraints, so the app removes child rows itself when a topic is deleted; adding foreign keys with cascade delete is listed in our future work.

4.3 DAO operations

DAO	Create	Read	Update	Delete
TopicDao	insertTopic	getAllTopics (Flow), getTopicById, getTopicByTitle, getTopicCount	updateTopic	deleteTopic, deleteTopicById
FlashcardDao	insertFlashcard	getAllFlashcards (Flow), getFlashcardById, getFlashcardCountForTopic	updateFlashcard	deleteFlashcardById, deleteFlashcardsForTopic
ResourceDao	insertResource	getAllResources (Flow)	updateResource	deleteResourceById, deleteResourcesForTopic
ReviewResultDao	insertResult	getAllResults (Flow), getResultsForTopic (Flow), answer and correct counts	– (history is append-only)	clearTopicResults
StudyPackDao	insertStudyPack	getAllStudyPacks (Flow), getStudyPackById	updateStudyPack	deleteStudyPackById

List queries return a Kotlin Flow, so the screens update automatically when a row changes. One-off reads and all writes are suspend functions that run in coroutines, off the main thread. Review results are deliberately append-only: an answer is a fact about the past, so we never edit it, we only add new ones or clear a topic's history.

4.4 Other local storage

Store	Contents
SharedPreferences loopi_auth	Name, email, salted PBKDF2 password hash and recovery-code hash, signed-in flag
SharedPreferences (settings)	Daily goal, reminder preference, default review mode, first-time setup flags
SharedPreferences loopi_community	Joined groups and the learner's own posts (JSON)
SharedPreferences loopi_support	Problem reports from the Report a Problem form (JSON)

5. Code Explanations

This section explains the most important parts of our code with short excerpts. We have removed imports and some formatting to keep them readable, and added comments that explain each step. The full source is in the submitted Android Studio project and selected files are listed in Appendix A.

5.1 Architecture overview

Loopi has three layers. The **UI layer** is made of Composable screen functions in the Flow files. The **state layer** is a set of plain Kotlin state holder classes (LoopiAppState, Flow3AppState, Flow4AppState, Flow6AppState and CommunityState). They are created once in MainActivity, read data from Room as Flows, expose it as Compose state, and contain the actions that write to the database. The **data layer** is Room (entities, DAOs and LoopiDatabase) plus SharedPreferences. Screens never talk to the DAOs directly; they call functions on a state holder.

Our state holders play the role that a ViewModel plays in the rubric: they keep UI logic and data logic out of the composables. They are not AndroidX ViewModel subclasses, which means they are recreated if the activity is recreated. We explain this honestly in Section 8 and plan to move them to ViewModel and StateFlow as the next step.

5.2 Room entity and DAO

```
@Entity(
    tableName = "flashcards",
    indices = [Index(value = ["topicId"])]      // fast lookup of a topic's cards
)
data class FlashcardEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val topicId: Int,                           // the topic this card belongs to
    val question: String,
    val answer: String,
    @ColumnInfo(defaultValue = "") val explanation: String = "",
    // Wrong options kept with the card (one per line), e.g. from PDF-generated questions
    @ColumnInfo(defaultValue = "") val wrongOptions: String = "",
    val createdAt: Long = System.currentTimeMillis(),
    val updatedAt: Long = System.currentTimeMillis()
)

@Dao
interface FlashcardDao {
    @Query("SELECT * FROM flashcards ORDER BY updatedAt DESC")
    fun getAllFlashcards(): Flow<List<FlashcardEntity>>     // observed by the UI

    @Insert suspend fun insertFlashcard(flashcard: FlashcardEntity): Long
    @Update suspend fun updateFlashcard(flashcard: FlashcardEntity)

    @Query("DELETE FROM flashcards WHERE id = :id")
    suspend fun deleteFlashcardById(id: Int)

    @Query("DELETE FROM flashcards WHERE topicId = :topicId")
    suspend fun deleteFlashcardsForTopic(topicId: Int)      // used when a topic is deleted
}
```

Listing 1. FlashcardEntity.kt and FlashcardDao.kt

5.3 Database class and migration

```
@Database(
    entities = [TopicEntity::class, FlashcardEntity::class, ResourceEntity::class,
                ReviewResultEntity::class, StudyPackEntity::class],
    version = 8,
    exportSchema = true
)
abstract class LoopiDatabase : RoomDatabase() {
    abstract fun topicDao(): TopicDao
    abstract fun flashcardDao(): FlashcardDao
    abstract fun resourceDao(): ResourceDao
    abstract fun reviewResultDao(): ReviewResultDao
    abstract fun studyPackDao(): StudyPackDao

    companion object {
        @Volatile private var INSTANCE: LoopiDatabase? = null

        // Version 8 adds wrongOptions so PDF questions keep their own distractors
        private val MIGRATION_7_8 = object : Migration(7, 8) {
            override fun migrate(database: SupportSQLiteDatabase) {
                database.execSQL("ALTER TABLE `flashcards` ADD COLUMN `wrongOptions` TEXT
NOT NULL DEFAULT ''")
            }
        }

        // One shared instance for the whole app (singleton)
        fun getDatabase(context: Context): LoopiDatabase =
            INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(context.applicationContext,
                        LoopiDatabase::class.java, "loopi_database")
                    .addMigrations(MIGRATION_1_2, MIGRATION_2_3, MIGRATION_3_4,
                        MIGRATION_4_5, MIGRATION_5_6, MIGRATION_6_7, MIGRATION_7_8)
                    .build()
                    .also { INSTANCE = it }
            }
    }
}
```

Listing 2. LoopiDatabase.kt (abbreviated)

5.4 Navigation with arguments

```
@Composable
fun LoopiNavigation(navController: NavHostController, appState: LoopiAppState,
                    flow3State: Flow3AppState, flow4State: Flow4AppState,
                    flow6State: Flow6AppState, community: CommunityState) {
    NavHost(navController = navController, startDestination = "splash") {
        composable("home") { /* Flow2 Home dashboard */ }
        composable("topics") { /* Study Topics list */ }

        // The topic id is passed as an Int argument in the route
        composable(
            route = "edit_topic/{topicId}",
            arguments = listOf(navArgument("topicId") { type = NavType.IntType })
        ) { backStackEntry ->
            val topicId = backStackEntry.arguments?.getInt("topicId") ?: -1
```

```
            EditTopicScreen(navController = navController, appState = appState, topicId =
topicId)
        }

        composable("progress") { /* Flow5ProgressScreen */ }
        composable("community") { CommunityScreen(navController, community) }
        composable("help_support") { HelpSupportScreen(navController) }
        // ... more than 50 routes in total
    }
}
```

Listing 3. MainActivity.kt – NavHost and an argument route (abbreviated)

Navigating is a single call such as `navController.navigate("review_question/$topicId")`. When a review ends we use `popUpTo("review_question/$topicId") { inclusive = true }` so that pressing Back from the results screen does not return to an answered question. The bottom navigation bar (LoopiBottomNavigation) switches between the five main tabs.

5.5 Quiz: confidence before the answer

The quiz screen decides what the button does purely from state. Until the learner has chosen both an answer and a confidence level, the button is disabled and tells them what is missing.

```
// Answer and confidence are chosen first; the result is revealed only after "Check
answer".
val revealed = flow4State.lastAnswer?.flashcardId == card.id

Button(
    onClick = {
        if (!revealed) {
            flow4State.commitCurrentAnswer()          // save the answer, then reveal
        } else if (!flow4State.moveToNextQuestion()) {
            navController.navigate("review_complete/$topicId") {
                popUpTo("review_question/$topicId") { inclusive = true }
            }
        }
    },
    // Only enabled once both an answer and a confidence are selected
    enabled = revealed ||
        (flow4State.selectedAnswer != null && flow4State.selectedConfidence != null)
) {
    Text(when {
        !revealed && flow4State.selectedAnswer == null     -> "Choose an answer"
        !revealed && flow4State.selectedConfidence == null -> "Rate your confidence"
        !revealed -> "Check answer"
        isLast    -> "See results"
        else      -> "Next question"
    })
}
```

Listing 4. Flow4Screens.kt – Check answer button driven by state

When the learner checks an answer, commitCurrentAnswer() compares it with the card, saves a ReviewResultEntity and updates the topic's progress inside one database transaction, so the topic percentage on Home and Topics is always in step with the saved answers.

```
val correct = answer.trim().equals(card.answer.trim(), ignoreCase = true)
```

```
scope.launch {
    database.withTransaction {                          // both writes succeed or neither
does
        reviewDao.insertResult(
            ReviewResultEntity(
                topicId = topicId, flashcardId = card.id,
                question = card.question, correctAnswer = card.answer,
                explanation = card.explanation, userAnswer = answer,
                isCorrect = correct, confidence = confidence,
                // elapsedRealtime is not affected by clock changes
                responseTimeMillis = (SystemClock.elapsedRealtime() -
questionStartedRealtime)
                    .coerceAtLeast(0L)
            )
        )
        // Recalculate the topic's accuracy from all of its saved answers
        val attempts = reviewDao.getAnswerCountForTopic(topicId)
        val correctAnswers = reviewDao.getCorrectAnswerCountForTopic(topicId)
        topicDao.getTopicById(topicId)?.let { topic ->
            topicDao.updateTopic(topic.copy(
                progress = if (attempts == 0) 0 else correctAnswers * 100 / attempts,
                lastUpdated = System.currentTimeMillis()))
        }
    }
}
```

Listing 5. Flow4AppState.commitCurrentAnswer() (abbreviated)

```
/** Compares how sure the learner said they were with how they actually did. */
internal fun calibrationMessage(correct: Boolean, confidence: String): String = when {
    correct && confidence == "I know it" -> "You were sure and you were right. Well
calibrated."
    correct && confidence == "Not sure"  -> "You knew more than you thought. Trust yourself
a little more."
    correct                              -> "Right answer. One more review will make it
stick."
    confidence == "I know it"            -> "You felt sure, but this one needs another
look. Added to your weak spots."
    else                                 -> "Good call flagging this. It's been added to
your weak spots."
}
```

Listing 6. Flow4Screens.kt – personal feedback after each answer

5.6 Confidence calibration score

The calibration score on the Progress screen is plain Kotlin, so we could unit test it (CalibrationTest) without the UI.

```
internal data class Calibration(val score: Int, val overconfident: Int,
                                val underconfident: Int, val rated: Int)

/**
 * How well confidence ratings match real results: "I know it" should be right and
 * "Not sure" should be wrong. "Maybe" counts as half a match either way.
 */
internal fun calculateCalibration(results: List<ReviewResultEntity>): Calibration? {
```

```
    val rated = results.filter { it.confidence in setOf("I know it", "Maybe", "Not sure") }
    if (rated.isEmpty()) return null                    // the UI shows a friendly empty
state
    val points = rated.sumOf { record ->
        when (record.confidence) {
            "I know it" -> if (record.isCorrect) 1.0 else 0.0
            "Not sure"  -> if (record.isCorrect) 0.0 else 1.0
            else        -> 0.5
        }
    }
    return Calibration(
        score = (points * 100 / rated.size).roundToInt(),
        overconfident  = rated.count { it.confidence == "I know it" && !it.isCorrect },
        underconfident = rated.count { it.confidence == "Not sure" && it.isCorrect },
        rated = rated.size
    )
}
```

Listing 7. Flow5Screens.kt – calculateCalibration()

5.7 PDF to multiple-choice questions, on the device

```
private const val MAX_PDF_PAGES = 40

/** Pulls the text layer out of a PDF. Scanned PDFs have no text layer and return blank. */
internal suspend fun extractPdfText(context: Context, uri: Uri): String =
    withContext(Dispatchers.IO) {                       // file work off the main thread
        PDFBoxResourceLoader.init(context.applicationContext)
        val input = context.contentResolver.openInputStream(uri) ?: error("Could not open
that file.")
        input.use { stream ->
            PDDocument.load(stream).use { document ->
                PDFTextStripper().apply {
                    startPage = 1
                    endPage = minOf(document.numberOfPages, MAX_PDF_PAGES)
                }.getText(document)
            }
        }
    }
```

Listing 8. PdfQuizScreen.kt – reading a PDF offline with PdfBox-Android

```
object McqGenerator {
    // "Normalization is the process of ..." -> term + description
    private val definitionPattern = Regex(
        "^(?:An?|The)?\\s*([A-Za-z][A-Za-z0-9()/\\- ]{1,48}?)\\s+" +
        "(?:is|are|refers to|means|is defined as|is known as|is called|describes)\\s+(.
{15,})$")

    fun generate(rawText: String, maxQuestions: Int = 15): List<GeneratedQuestion> {
        val text = cleanText(rawText)
        if (text.length < 80) return emptyList()       // not enough text to work with
        // 1. find definitions in sentences and "Term: description" lines
        // 2. collect key terms: defined terms, repeated capitalised phrases, frequent
words
        // 3. definitions -> "Which term matches this description"; other sentences -> fill
in the blank
        ...
```

```
        definitions.forEach { (term, value) ->
            val (description, source) = value
            accept(GeneratedQuestion(
                question = "Which term matches this description: \"$
{description.trimEnd('.')}\"?",
                answer = term,
                distractors = pickDistractors(term, terms),   // other terms from the same
PDF
                explanation = source))
        }
        return questions.take(maxQuestions)            // definition questions come first
    }

    private fun pickDistractors(answer: String, terms: List<String>): List<String> =
        terms.filter { !it.equals(answer, ignoreCase = true) }
             .filter { !it.contains(answer, true) && !answer.contains(it, true) }
             // prefer terms of a similar shape so the right answer doesn't stand out
             .sortedBy { abs(it.split(" ").size - answer.split(" ").size) * 10 +
                         abs(it.length - answer.length) }
             .take(3)
}
```

Listing 9. McqGenerator.kt – heuristic question generation (abbreviated)

```
/** Rough category from words in the notes, matching the Add Topic categories. */
fun suggestCategory(rawText: String): String {
    val text = rawText.lowercase()
    val scores = mapOf(
        "Computer Science" to listOf("algorithm", "process", "memory", "program",
"database", "software"),
        "Information Technology" to listOf("network", "server", "security", "firewall",
"protocol", "router"),
        "Business" to listOf("market", "customer", "profit", "finance", "management"),
        "Mathematics" to listOf("equation", "theorem", "matrix", "integral", "probability")
        // ... Engineering keywords omitted
    ).mapValues { (_, words) -> words.sumOf { word ->
Regex(Regex.escape(word)).findAll(text).count() } }
    // The best-scoring category wins, but only if it appears at least twice
    return scores.maxByOrNull { it.value }?.takeIf { it.value >= 2 }?.key ?: "Other"
}
```

Listing 10. McqGenerator.kt – suggestCategory() used by New Topic from PDF (abbreviated)

When the learner saves the reviewed questions, each one becomes a FlashcardEntity whose wrongOptions column stores the three distractors. The quiz then uses those options first, and only falls back to answers from other cards in the topic when a card has none.

5.8 Study pack export

```
/** Builds a v2 export that carries the topics' flashcards,
    so a pack works on a device that has none of them. */
suspend fun buildExport(pack: StudyPackEntity): String {
    val names = packTopicNames(pack)                    // topic titles saved in the pack
    val cards = flashcardDao.getAllFlashcards().first()
    val topicData = JSONArray()
    names.forEach { title ->
        val topic = topicDao.getTopicByTitle(title) ?: return@forEach
        val topicCards = JSONArray()
```

```
        cards.filter { it.topicId == topic.id }.forEach { card ->
            topicCards.put(JSONObject()
                .put("question", card.question)
                .put("answer", card.answer)
                .put("explanation", card.explanation))
        }
        topicData.put(JSONObject()
            .put("title", topic.title).put("description", topic.description)
            .put("category", topic.category).put("difficulty", topic.difficulty)
            .put("cards", topicCards))
    }
    return JSONObject()
        .put("type", "loopi_study_pack").put("version", 2)   // lets import check the
format
        .put("name", pack.name).put("description", pack.description)
        .put("topics", JSONArray(names)).put("topicData", topicData)
        .put("exportedAt", System.currentTimeMillis())
        .toString(2)
}
```

Listing 11. Flow6Screens.kt – Flow6AppState.buildExport()

The file is written with the Storage Access Framework (`rememberLauncherForActivityResult(ActivityResultContracts.CreateDocument("application/json"))`), so the learner chooses where to save it and the app needs no storage permission. Import uses `OpenDocument`, checks that the file has a name and at least one topic, reuses topics that already exist with the same title, and shows how many topics and cards were added.

5.9 Input validation and user feedback

Every form keeps its field values and error flags in Compose state. Errors appear only after the learner presses Save, and disappear as soon as the field is fixed. Each OutlinedTextField sets `isError` and shows a supporting message under the field. After a successful save, update, delete, export or import, the app shows a Snackbar or a status message.

Form	Input types	Validation
Sign Up / Sign In	TextField, password field with eye toggle	Name, email format, password length, matching passwords, wrong password
Add / Edit Topic	TextField, DropdownMenu, difficulty choice, number field	Title, description, category and number of cards are required; number must be valid
Flashcard and Resource editors	TextField, DropdownMenu (resource type)	Question and answer required; resource title required
Quiz	Option buttons, confidence buttons	Check answer is disabled until both are chosen
Create Pack	TextField, Checkbox list	Title required, at least one topic selected
Settings	Switch, RadioButton	Values saved immediately
Report a Problem	DropdownMenu, TextField	Problem type and description required

5.10 Testing

We tested in two ways. First, we wrote JUnit unit tests for the logic that must be correct: PasswordHashTest, DefaultStudyContentTest, HomeProgressTest, Flow5AnalyticsTest, CalibrationTest, StudyPackFormatTest and McqGeneratorTest. Together they contain 21 tests, and all of them pass on the final build. Second, we ran every flow by hand on an Android emulator (API 29 and above): creating, editing and deleting topics, cards and resources, finishing quizzes, checking that Progress updates, exporting a pack and importing it again, generating questions from a real lecture PDF, turning the network off, and closing and reopening the app to confirm that all data persists.

6. Collaboration and Task Allocation

We divided the work by flow, using the wireframe sheet in Figure 2, so that each member owned a complete part of the app from the database to the screens. After the core flows worked, we met to integrate them, agreed on one shared theme, and worked together on the redesign, the PDF feature and the Community and Help screens.

Member	Main responsibility	Contribution
Akhmetova Azhar (M1)	Flow 2: Home, Study Topics, Add/Edit Topic; sign-in review with M5; reminders	Home dashboard and daily-goal ring; topic list with search, filter and sort; Add Topic validation; TopicEntity and TopicDao; reminder and daily-goal settings
Tussupova Dariya (M2)	Flow 3: Topic detail, Flashcards, Resources	Topic detail hub and stat tiles; flashcard list and editor; flip-card study mode; resources for notes, links and PDFs; FlashcardEntity, ResourceEntity and their DAOs
Zhassyuzak Nursaule (M3)	Flow 4: Quiz, review and confidence	Review settings; one-screen quiz with confidence before reveal; answer options and wrong options; feedback and Explain simply; ReviewResultEntity and DAO; session results and weak-card practice
Suzain Mudabbir Sayyad (M4)	Flow 5: Progress and weak spots; Profile and Settings	Progress chart and period filter; weak-spot card; confidence calibration score; topic progress; Profile and Settings screens; analytics unit tests
Sanjeet Sachin Zele (M5)	Flow 6: Study packs; sign-in review with M1; offline support	Study pack list, create and detail screens; export/import format v2 with flashcards; StudyPackEntity and DAO; local PBKDF2 sign-in review; offline testing
All members	Integration and extensions	Shared Loopi theme and gradient redesign; PDF-to-MCQ generator and review screen; Community and Help & Support (Flows 7–8); testing on the emulator; this report and the demo

We kept the code in one Android Studio project under Git, and each member tested their own flow on the emulator before it was merged. When two flows depended on each other, for example Flow 4 writing review results that Flow 5 reads, the two owners agreed on the entity fields first so that both could work in parallel.

7. Reflection Report (Individual)

Each member answers the five reflection prompts from the brief based on their own role in the project.

7.1 Akhmetova Azhar (I26043015)

Role: Flow 2: Home dashboard, Study Topics and Add Topic; reviewed onboarding and sign-in with Sanjeet; reminder and notification settings.

1. How closely does the final app match our Assignment 1 proposal? What changed and why?

When I compare the Loopi on my phone today with the blueprint page we wrote for Assignment 1, I can honestly say the core idea survived almost untouched. We promised a simple study app where a university student creates topics, adds flashcards or tests, rates confidence before seeing the answer, and can see which topics need more attention. Every one of those promises is now a working screen. The part I owned, the Home dashboard and the topic library, is the first thing a learner sees, so for me it was the place where the proposal had to feel real.

Some things did change. In Assignment 1 our slogan was "Learn. Practice. Remember." and during the build we changed it to "Learn. Review. Repeat." because the second version describes what the app actually makes you do: you review a card, you see your result, and the app brings the weak cards back. The proposal also talked about the app "reminding" students when it is time to review. I built the reminder settings and the daily goal, but the app does not yet send scheduled push notifications. We decided that a correct, offline Room database and a reliable review flow were more important than notifications for this submission, and we would rather be honest that reminders are a saved preference than show something half working.

The biggest visible change is the design. Our first version of Home used plain white cards with emoji icons, and when we put it next to the wireframe it looked unfinished. In the final weeks we moved to one shared Loopi theme with a blue gradient header, a daily-goal ring that shows progress like 2/20, coloured quick-action tiles for Study, Add topic, Packs and Groups, and a "Topics due today" list. I think this is closer to what we imagined in Assignment 1 than our early screens were, even though the early screens technically "worked".

2. Which features from the proposal were most challenging to implement?

For my flow, the hardest part was not drawing the screens but making the numbers on them agree with each other. At one point Home said three cards were reviewed, Stats said one answer and Profile said four answers, because each screen counted something slightly different: cards seen, answers saved, or sessions finished. A student would immediately stop trusting the app if the dashboard and the progress page disagree. Fixing it meant going back to the review_results table and agreeing as a group on one definition of "reviewed", then using the same calculation everywhere. We added a unit test (HomeProgressTest) so that the rule could not silently break again.

Add Topic validation was also more work than I expected. The form has a title, description, a category dropdown, a difficulty choice and the number of cards. Each field needs its own error message ("Title is required", "Description is required", "Please select a category", "Number of cards is required"), the error must only appear after the user tries to save, and it must disappear as soon as they fix the field. Getting that behaviour right with Compose state taught me that validation is really a user experience problem, not only an if statement.

Finally, the sign-in review with Sanjeet was challenging because we had to decide what "login" even means in an offline app. Our proposal never promised a server, so we kept sign-up and sign-in local and stored the password as a salted PBKDF2 hash instead of plain text. Explaining that clearly to a user, without scaring them, is why the sign-in screen now says "Your account and study data stay on this phone. No internet needed."

3. Which Android development skills from this course did I apply, and how?

I used almost every topic from the course in my flow. The Home screen is built from Column, Row, Box and Card composables with Modifier for spacing, and the topic list is a LazyColumn where every item shows text plus an icon tile and a progress bar, and is tappable to open the topic detail screen. This directly matches requirement 3 of the brief.

For state I worked with remember and mutableStateOf inside the composables and with our shared LoopiAppState holder, which reads topics from Room as a Flow so the list updates when a topic is added or deleted. The Add Topic form uses OutlinedTextField, a DropdownMenu for category and RadioButton-style choices for difficulty, so it covers the "at least two input types" requirement. Feedback after saving or deleting is shown with a Snackbar.

I also used navigation arguments. Tapping a topic navigates to a route such as topic_detail/{topicId}, and editing goes to edit_topic/{topicId}, where the id is read from the back stack entry and used to load the topic from TopicDao. Finally, I worked with Room through TopicDao: insertTopic, updateTopic, deleteTopic and getAllTopics. Seeing a topic still there after closing and reopening the emulator was the moment the whole course clicked for me.

4. What would I do differently with more time or resources?

With more time, the first thing I would add is real scheduled reminders using WorkManager, so the "daily goal" setting actually sends a gentle notification in the evening. In our Assignment 1 survey students said they lose motivation, and I think a reminder at the right time is a small feature with a big effect.

Second, I would move our state holders into proper AndroidX ViewModel classes with StateFlow. Our LoopiAppState works, but it is created in the activity with remember, so it does not survive configuration changes as cleanly as a ViewModel would, and the rubric specifically mentions ViewModel. Now that I understand the difference, I would start with ViewModels from the first day instead of refactoring later.

Third, I would test the dashboard with real classmates, not only on the emulator. We designed Home based on our survey and the wireframe, but I would like to watch five students open the app for the first time and see where they tap. I suspect we would simplify the dashboard even more.

5. How has building this app changed my perspective as a future developer?

In my Assignment 1 reflection I wrote that a good educational app should be "simple, stable and easy to use" and that having a lot of features does not make an app better. Building Loopi proved that to me in a very practical way. Every extra number on the Home screen had to be correct, consistent with other screens and understandable, and each one cost us time. I now think of every element on a screen as a promise to the user.

I also learned that real development is mostly about the parts nobody sees: database migrations, validation states, edge cases like a topic with zero cards, and making sure data is still there after a restart. Before this course I thought of an app as its screens. Now I think of it as screens on top of data that must never be lost. As a future developer I want to keep starting from the learner's problem, as we

did in Assignment 1, but I will also plan the data and the architecture much earlier, because that is where most of our late changes came from.

7.2 Tussupova Dariya (I26043025)

Role: Flow 3: Topic Detail, Flashcards and Resources (notes, links and PDF files attached to a topic).

1. How closely does the final app match our Assignment 1 proposal? What changed and why?

In Assignment 1 we wrote that Loopi "saves learning materials on the device" and that students can create their own flashcards and tests. My part, Flow 3, is exactly that promise: a topic detail page, a flashcard list where cards can be added, edited and deleted, and a resources section where a learner can keep notes, links and PDF files for the topic. In the final app all of this works and is stored in Room, so I feel that my flow matches the proposal closely.

What changed most was the way a learner studies cards. In the wireframe, Flashcards (3B) showed a card with Not Sure, Maybe and I Know It buttons. Our first implementation only listed cards as text, which was useful for editing but not for studying. Later we added a real flip-card study mode: you see the question, tap to see the answer, and rate how well you know it, with Previous and Next buttons and a 1/10 progress bar. The topic detail page also gained stat tiles (flashcards, resources, difficulty and accuracy) and clear Start Studying and Take a Quiz buttons, which make the page feel like a hub instead of a list.

The other change is the PDF feature. In Assignment 1 we only said materials would be stored. In the final app a learner can choose a PDF of lecture notes and Loopi suggests multiple-choice questions from it, completely offline, and the PDF is also saved in the topic's resources. This was not in our proposal, but it grew naturally from my resources screen, because students kept asking "why do I have to type every card myself if I already have the slides?"

2. Which features from the proposal were most challenging to implement?

The most challenging part for me was the relationship between topics, flashcards and resources in the database. A flashcard and a resource both belong to a topic through a topicId column. When a topic is deleted, its cards and resources must be deleted too, otherwise they stay in the database as "orphans" and still appear in counts. Our entities do not declare Room foreign keys, so we had to add deleteFlashcardsForTopic and deleteResourcesForTopic to the DAOs and call them in the right order. I learned that a database design decision made in week one can still cause bugs in week ten.

Database migrations were also new and a bit frightening for me. Every time we added a column, for example the explanation for a flashcard or the list of wrong options for PDF-generated questions, we had to increase the database version and write a Migration with an ALTER TABLE statement. If we forgot, the app crashed on the emulator that already had the older database. By the end we had eight versions and seven migrations, and I understood why real apps are careful with schema changes.

Finally, opening files on Android was harder than I expected. A PDF picked by the user is a content Uri, not a normal file path, so we had to read it through the ContentResolver and keep only the Uri in the resource record.

3. Which Android development skills from this course did I apply, and how?

I applied Room more than any other topic: FlashcardEntity and ResourceEntity with @Entity, @PrimaryKey and an index on topicId, and DAO interfaces with @Insert, @Update, @Query for delete and Flow-returning queries for lists. Because the lists are Flows, adding a card on the edit screen immediately updates the flashcard list and the count on the topic detail page.

I used LazyColumn for the flashcard and resource lists, where each item has text plus an icon (for example a PDF icon or a link icon) and opens a detail or edit screen when tapped. For input I used text

fields for question, answer and explanation, and a dropdown for the resource type, with validation so a card cannot be saved without a question and an answer. I also used navigation arguments to pass the topicId and the flashcardId between the topic detail, list and edit screens.

For the flip card I used Compose state: a remembered value decides whether the question or the answer side is showing, and tapping the card changes it. It is a small screen, but it made me understand how state drives the UI in Compose instead of me changing views manually.

4. What would I do differently with more time or resources?

With more time I would add proper foreign keys with onDelete = CASCADE, so the database itself guarantees that cards and resources are removed with their topic. This is cleaner than remembering to call several delete functions.

I would also like to support scanned PDFs. At the moment Loopi reads only PDFs that have a text layer; a photo of a page shows a clear message that the text cannot be read. With ML Kit text recognition, which also works offline, we could read scanned notes too. I would also add a preview of the PDF inside the app, instead of only storing it as a resource.

Lastly, I would add a spaced-repetition schedule for each card, so the flip-card mode shows the cards that are due today first. In Assignment 1 we promised that the app would "bring them for review over time", and at the moment our weak-spot logic is based on accuracy rather than a real schedule.

5. How has building this app changed my perspective as a future developer?

In Assignment 1 I wrote that "it would be much easier to start with simple features first" and that AI and other advanced ideas "take a lot of work". Building Loopi confirmed this. Even a simple flashcard involves an entity, a DAO, a migration, a list screen, an edit form, validation and navigation. I now respect how much work is hidden behind a feature that users see as one button.

I also learned to think about the user's real effort. The PDF-to-quiz feature started from a simple observation: students already have their material and do not want to type it again. As a developer I want to keep asking "what does the learner already have, and how can I save them time?" rather than adding features because they sound modern. This project made me more patient, more careful with data, and more confident that I can build something real on Android.

7.3 Zhassyuzak Nursaule (I26043027)

Role: Flow 4: Quiz, review session, answer feedback and confidence rating.

1. How closely does the final app match our Assignment 1 proposal? What changed and why?

The heart of our Assignment 1 proposal was this sentence: "Before seeing the correct answer, students say how confident they feel. Then they can compare their confidence with the actual result, read a short explanation and try again." Flow 4, which I was responsible for, is where that sentence becomes the app. In the final version, a learner chooses an answer, chooses Not sure, Maybe or I know it, presses Check answer, and only then sees whether they were right, with a short "Explain simply" box and the correct answer. So I can say the final quiz matches our proposal very closely.

However, it did not start that way. Our first working version split one question into four screens: the question, then a Correct or Incorrect screen, then an explanation screen, then a separate confidence screen. It technically saved everything, but it had two problems. First, confidence was asked after the answer was revealed, which is the opposite of what we proposed and makes the confidence rating almost meaningless. Second, four screens per card felt slow; ten cards meant forty screens. We redesigned it into one screen where the answer and confidence are chosen first and the result appears in place. This was the single most important change in the whole project for me, because it brought the app back to our original research idea.

We also added one thing we did not propose: a personal message after each answer that compares confidence with the result, for example "You felt sure, but this one needs another look. Added to your weak spots." This came from our survey, where students said they want feedback that is simple and personal.

2. Which features from the proposal were most challenging to implement?

The most challenging part was managing state across a review session. The quiz needs to know the list of cards for the session, the current index, the selected answer, the selected confidence, whether the current card has been revealed, and the time when the question appeared. It must also work in practice mode for weak cards. If any one of these is wrong, the user sees the wrong card or the button does the wrong thing. We solved this with a Flow4AppState class that keeps the session state and a single commitCurrentAnswer function that saves the result.

Building the answer options was also tricky. A multiple-choice question needs one correct answer and three wrong options that are believable. For normal flashcards we take wrong options from other cards in the same topic. For PDF-generated cards, the generator already gives three wrong options, so we store them in the new wrongOptions column and use them first. Making sure the options never repeat the correct answer and are shuffled took several rounds of testing.

Measuring response time correctly was the last challenge. We first used the wall clock, but it can change. We switched to SystemClock.elapsedRealtime(), which is meant for measuring durations.

3. Which Android development skills from this course did I apply, and how?

I applied Compose state heavily. The Check answer button is enabled only when both an answer and a confidence are selected, and its label changes from "Choose an answer" to "Rate your confidence" to "Check answer" to "Next question". All of this is just state read by the composable, which showed me how declarative UI really works.

I used Room through ReviewResultDao. Each submitted answer becomes a ReviewResultEntity row with the question, the user's answer, the correct answer, whether it was correct, the confidence, the

response time and the timestamp. The insert and the topic progress update run together inside database.withTransaction, so the topic's accuracy is always in step with its saved answers.

I also used navigation with arguments: review_question/{topicId}, review_complete/{topicId} and a practice route for weak cards, and popUpTo so that pressing Back after finishing does not return the learner to a question they already answered. Material 3 components such as Scaffold, Surface, Button and Card give the quiz its structure.

4. What would I do differently with more time or resources?

With more time I would add more question types: true or false, typing the answer, and matching. At the moment almost all questions are multiple choice. Typing the answer would be a stronger test of memory, although it needs more flexible checking of the user's text.

I would also connect confidence to scheduling. Right now confidence is used for feedback and for the calibration score in Progress, but a card answered correctly with "Not sure" should probably come back sooner than one answered correctly with "I know it". That would make our proposal's idea of reviewing "over time" much stronger.

Finally, I would write instrumented UI tests for the quiz. We have unit tests for the calibration and the generator, but the quiz flow was tested manually on the emulator. Automated tests would have saved us time when we redesigned it.

5. How has building this app changed my perspective as a future developer?

In my Assignment 1 reflection I wrote that I used to be interested in all the features an app could have, but research made me ask why each feature exists. Redesigning the quiz showed me this in practice. The four-screen version had every feature, but it put them in the wrong order and lost the main idea. Changing the order of two steps mattered more than adding anything new.

I also understood that educational apps deal with how people feel about learning. A message like "You knew more than you thought" is a small piece of text, but it can change how a student feels about a hard topic. As a future developer I want to pay attention to these details, to test with real users, and to always go back to the problem we wrote down at the beginning when I am unsure what to build next.

7.4 Suzain Mudabbir Sayyad (I26043300)

Role: Flow 5: Progress, weak spots and confidence calibration; Profile and Settings screens.

1. How closely does the final app match our Assignment 1 proposal? What changed and why?

Our Assignment 1 proposal said that Loopi "shows progress by topic" and "shows which topic they need to pay attention more". My part, Flow 5, is responsible for exactly this, together with the Profile and Settings screens. In the final app the Progress tab shows accuracy per answer as a bar chart for the week, month or all time, the number of correct and incorrect answers, the average answer time, how many topics were practised, a weak-spot card with a Practice button, and a confidence calibration score. Profile shows topics, cards and overall accuracy, the study streak, and links to study packs, community, settings and help. So my flow covers the proposal and goes a little further.

The calibration score is the main addition. In the proposal we only said that students compare their confidence with the result for each question. While building Progress I realised we already save the confidence and the correctness of every answer, so we could summarise how well a student's confidence matches reality across many answers. Now the app can say, for example, "You were sure but wrong 1 time(s). Slow down on cards that feel easy." I think this is the most original part of Loopi and it comes directly from our research problem: students who feel confident but still make mistakes.

One thing that changed is that Progress has no cloud history. Everything is calculated on the phone from the review_results table. This matches our offline-first proposal, but it means a student's progress does not move with them to a new phone, which I explain honestly in Settings and Help.

2. Which features from the proposal were most challenging to implement?

The hardest part was making the statistics correct and consistent. Accuracy, correct and incorrect counts, weak spots and the Home dashboard all come from the same saved answers, but at first they used different filters and periods, so they showed different numbers. I had to move the calculations into small functions such as calculateAccuracy, accuracyBuckets and calculateCalibration, use them everywhere, and write unit tests (Flow5AnalyticsTest and CalibrationTest) to check them with known data.

Designing the calibration formula was also a challenge, because I had to decide what "well calibrated" means. In the end, an "I know it" answer earns a point if it is correct, a "Not sure" answer earns a point if it is wrong, and "Maybe" earns half a point either way. It is simple enough to explain in the Help screen and still tells students something useful. I also had to handle empty data, so a new user sees a friendly message instead of "0%" or a crash.

Personally, a practical challenge was the same one I had in Assignment 1: I do not have an Android phone, so I tested everything on an Android Studio virtual device. This made me careful about different screen sizes and about checking that data stayed after closing and reopening the emulator.

3. Which Android development skills from this course did I apply, and how?

I applied Kotlin functions and data classes the most. The Calibration and AccuracyBucket data classes and the analytics functions are plain Kotlin that can be unit tested without the UI. This separation made my code easier to test and to reuse on Home and Profile.

In Compose I built the Progress screen from Column, Row, Card and Box, used a period selector with Week, Month and All time chips, drew the bars with gradient brushes, and used LoopiProgressBar for the calibration score. The data comes from Room as a Flow through ReviewResultDao.getAllResults(), so the chart updates right after a quiz.

In Settings I used Switch and RadioButton controls for the daily goal, reminders and the default review mode (weak cards only or all cards), with the values saved in SharedPreferences so they persist between launches. The Profile screen has an edit form with validation for the name and student ID. Navigation from Progress to a topic's progress page passes the topicId as an argument.

4. What would I do differently with more time or resources?

With more time I would build a longer history view, for example accuracy per topic over several weeks, and show a trend line, so students can see if they are improving rather than only their current state. I would also add an export of progress as a file, similar to study packs, so a student could back up their history.

I would like to make the calibration score smarter by weighting recent answers more, and by showing calibration per topic. A student could be well calibrated in Databases but overconfident in Networks, and that is useful to know before an exam.

I would also make the screens more accessible: checking colour contrast of the chart, adding content descriptions for screen readers, and testing with larger font sizes. And of course I would like to test the app on a real Android phone.

5. How has building this app changed my perspective as a future developer?

In Assignment 1 my reflection was mostly about collecting evidence: exploring Seekho, Master and Duolingo on a virtual device and comparing what I saw with what the apps claimed. Building Loopi taught me the other side. Now I am the one who has to make sure the numbers on the screen are true. If my weak-spot card says a topic is at 0%, a student may spend their evening on it, so that number must be correct.

This changed how I see myself as a developer. I used to think that analytics is about nice charts. Now I think it is about trust: clear definitions, tested calculations and honest messages when there is not enough data. I also learned that small, testable functions make a big difference when the team changes something late in the project. I want to keep this habit of testing the logic behind the UI in every app I build.

7.5 Sanjeet Sachin Zele (I26043274)

Role: Flow 6: Study Packs, including export and import; reviewed onboarding and sign-in with Azhar; offline support.

1. How closely does the final app match our Assignment 1 proposal? What changed and why?

In Assignment 1 we wrote that students can continue their practice without an internet connection and that "sharing flashcard sets with classmates could be an additional feature". My responsibilities were the study packs and offline support, so I was responsible for making both promises true without a server. In the final app a learner can create a study pack, give it a name and description, choose topics from a checklist, and then export the pack as a .json file or share it through Android's share sheet. A classmate can import that file and gets the topics and all their flashcards. Nothing is uploaded anywhere, and all the main features work in airplane mode.

The biggest change was in what a pack contains. Our first version only saved the pack name and the list of topic titles. That was fine on my own emulator, but when the pack was imported on another device that did not have those topics, the pack was empty. We changed the export format to version 2, which includes each topic's description, category, difficulty and flashcards. Now a pack really can be shared with a classmate who has never created those topics.

We also decided not to add Google or Apple sign-in even though they appear in the wireframe, because our proposal never asked for an online account, and adding them would have required a backend. Instead the sign-in screen tells the user that their data stays on the phone.

2. Which features from the proposal were most challenging to implement?

The most challenging part was designing the import. An imported file comes from outside the app, so it can be empty, broken, from an older version, or contain topics that already exist. We had to validate it: the pack must have a name and at least one topic, otherwise the user gets a clear message such as "The imported pack has no topics." If a topic with the same title already exists, we reuse it instead of creating a duplicate. At the end the app shows a summary, for example that a pack was imported with one new topic and four cards. Writing a unit test for the pack format (StudyPackFormatTest) helped me catch mistakes in the JSON.

Working with Android file pickers was also new. We used the Storage Access Framework through rememberLauncherForActivityResult with CreateDocument for export and OpenDocument for import, so the user chooses where the file goes, and the app does not need broad storage permissions.

Reviewing the local sign-in with Azhar was a challenge of a different kind. We had to make sure passwords were never stored as readable text, so we used a salted PBKDF2 hash, and we tested that a wrong password is rejected and that the reset flow uses a recovery code.

3. Which Android development skills from this course did I apply, and how?

I applied Room through StudyPackEntity and StudyPackDao, with insert, update, delete and Flow queries, and I read the topics and flashcards through their DAOs to build the export. The study pack list is a LazyColumn where each item shows the pack name, the number of topics and an icon, and tapping it opens the pack detail screen using the pack id as a navigation argument.

The Create Pack screen uses a text field for the title with validation, a description field, and a list of Checkbox items for choosing topics, so it uses several input types. After saving, exporting or importing, a status message gives feedback to the user. I also used Kotlin coroutines and suspend functions, because building an export reads from the database and must not block the UI.

For offline support, I checked every flow with the network turned off on the emulator. Because all data is in Room and SharedPreferences, and even the PDF reading is done on the device with PdfBox-Android, the app does not need the internet at all.

4. What would I do differently with more time or resources?

With more time I would add an optional sync, for example a small server or Firebase, but only if users want it. This would let a student keep their progress when they change phones and would allow the Shared tab of study packs to show packs from classmates directly. I would still keep offline as the default, because our research showed that students want less dependence on the internet.

I would also add pack versioning, so if a classmate updates their pack, I can import the new version and only the new cards are added. And I would like to add QR-code sharing, so two students sitting next to each other could share a pack without sending files.

Lastly, I would take more care to document my code earlier. I wrote clear comments only at the end, and it would have helped my teammates to understand the pack format sooner.

5. How has building this app changed my perspective as a future developer?

In Assignment 1 I wrote that I learned it is better to write about an app based on what I actually checked instead of assuming things. This project taught me the same lesson as a developer. A pack that works on my emulator is not a pack that works for a classmate; I had to test it on a clean device to see the real problem. Now I always ask "what happens on someone else's phone?"

I also understand much better why offline-first apps are harder than they look. Without a server, the app itself must handle sharing, validation, duplicates and privacy. This gave me more respect for data design and made me more confident with Kotlin, Room and Android's file system. I think these are skills I will use in every app I build in the future.

8. Limitations and Future Work

We want to be clear about what the current version does not do yet:

- **ViewModel.** Our state holders separate UI and data logic, but they are plain classes created in MainActivity, not AndroidX ViewModel with StateFlow. Moving them to ViewModel is our first planned change, so state survives configuration changes.
- **Reminders.** The daily goal and reminder preference are saved, but scheduled notifications (for example with WorkManager) are not yet sent.
- **Spaced repetition.** Topics due today and weak-card practice use accuracy and confidence; a per-card review schedule is future work.
- **Foreign keys.** Child rows are removed by the app when a topic is deleted; Room @ForeignKey with cascade delete would make this automatic.
- **PDF questions.** Generated questions are rule-based, so they work best on notes with clear definitions; scanned PDFs cannot be read without OCR (ML Kit text recognition could add this offline).
- **No sync.** There is no server, so accounts, community posts and progress stay on one phone. Google/Apple sign-in and email verification from the wireframe would need a backend.
- **Community** is local only and clearly labelled as such in the app.

9. References

[1] Group 6. (2026). Assignment 1: Decoding the Educational App Ecosystem – Research, Analysis & App Proposal (Loopi). IBM4203, August 2026. Internal coursework document.

[2] Group 6. (2026). Loopi App Wireframes & Feature Breakdown. Internal planning document.

[3] Android Developers. (2026). Jetpack Compose. https://developer.android.com/compose

[4] Android Developers. (2026). Save data in a local database using Room. https://developer.android.com/training/data-storage/room

[5] Android Developers. (2026). Navigation with Compose. https://developer.android.com/develop/ui/compose/navigation

[6] Android Developers. (2026). ViewModel overview. https://developer.android.com/topic/libraries/architecture/viewmodel

[7] Android Developers. (2026). Open files using the Storage Access Framework. https://developer.android.com/training/data-storage/shared/documents-files

[8] Google. (2026). Material Design 3. https://m3.material.io/

[9] Roush, T. (2024). PdfBox-Android. https://github.com/TomRoush/PdfBox-Android

[10] Duolingo. (2022). The Duolingo Streak Uses Habit Research to Keep You Motivated. https://blog.duolingo.com/how-duolingo-streak-builds-habit/

[11] Duolingo. (2020). Learning how to help you learn: Introducing Birdbrain! https://blog.duolingo.com/learning-how-to-help-you-learn-introducing-birdbrain/

[12] Google Play. (2026). Kahoot! Play & Create Quizzes. https://play.google.com/store/apps/details?id=no.mobitroll.kahoot.android

[13] Google Play. (2026). Duolingo: Language Lessons. https://play.google.com/store/apps/details?id=com.duolingo

[14] Bond, M., Khosravi, H., De Laat, M., & Bergdahl, N. (2024). A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour. International Journal of Educational Technology in Higher Education, 21, 4. https://doi.org/10.1186/s41239-023-00436-z

[15] UNESCO. (2023). Guidance for generative AI in education and research. https://www.unesco.org/en/articles/guidance-generative-ai-education-and-research

Appendix A. Code Listings and File Map

The complete source code is in the submitted Android Studio project (Loopi_Final.zip). The table below lists the files that make up each part of the app.

Concern	Files
Entry point and navigation	MainActivity.kt (LoopiNavigation, NavHost, routes)
Theme and shared components	ui/theme/Color.kt, Theme.kt, Type.kt; LoopiComponents.kt (LoopiHero, QuickActionTile, TopicIconTile, LoopiProgressBar, LoopiBottomNavigation)
Flow 1 – Onboarding and sign-in	flow1screens.kt
Flow 2 – Home and topics	flow2screens.kt (LoopiAppState), TopicEntity.kt, TopicDao.kt, DefaultStudyContent.kt
Flow 3 – Topic, cards, resources	Flow3Screens.kt (Flow3AppState), Flow3StudyScreen.kt, FlashcardEntity.kt, FlashcardDao.kt, ResourceEntity.kt, ResourceDao.kt
PDF to quiz	PdfQuizScreen.kt, McqGenerator.kt
Flow 4 – Quiz and confidence	Flow4Screens.kt (Flow4AppState), ReviewResultEntity.kt, ReviewResultDao.kt
Flow 5 – Progress, Profile, Settings	Flow5Screens.kt
Flow 6 – Study packs	Flow6Screens.kt (Flow6AppState), StudyPackEntity.kt, StudyPackDao.kt
Flows 7–8 – Community and Help	Flow78Screens.kt (CommunityState, HelpSupportScreen, ReportProblemScreen)
Database	LoopiDatabase.kt (version 8, migrations 1→8)
Unit tests	PasswordHashTest, DefaultStudyContentTest, HomeProgressTest, Flow5AnalyticsTest, CalibrationTest, StudyPackFormatTest, McqGeneratorTest

A.1 TopicEntity and TopicDao

```
@Entity(tableName = "topics")
data class TopicEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val description: String,
    val category: String,
    val difficulty: String,
    val numberOfCards: Int,
    @ColumnInfo(defaultValue = "0") val targetCardCount: Int = 0,
    val progress: Int = 0,                          // accuracy %, updated after each answer
    val lastUpdated: Long = System.currentTimeMillis()
)

@Dao
interface TopicDao {
    @Query("SELECT * FROM topics ORDER BY id ASC")
    fun getAllTopics(): Flow<List<TopicEntity>>

    @Query("SELECT * FROM topics WHERE id = :topicId LIMIT 1")
    suspend fun getTopicById(topicId: Int): TopicEntity?
```

```
    @Query("SELECT * FROM topics WHERE title = :title COLLATE NOCASE LIMIT 1")
    suspend fun getTopicByTitle(title: String): TopicEntity?   // used by pack
import/export

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTopic(topic: TopicEntity): Long

    @Update suspend fun updateTopic(topic: TopicEntity)
    @Delete suspend fun deleteTopic(topic: TopicEntity)

    @Query("DELETE FROM topics WHERE id = :topicId")
    suspend fun deleteTopicById(topicId: Int)

    @Query("SELECT COUNT(*) FROM topics")
    suspend fun getTopicCount(): Int
}
```

Listing A1. TopicEntity.kt and TopicDao.kt

A.2 ReviewResultEntity and ReviewResultDao

```
@Entity(tableName = "review_results",
        indices = [Index(value = ["topicId"]), Index(value = ["flashcardId"])])
data class ReviewResultEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val topicId: Int,
    val flashcardId: Int,
    val question: String,
    val correctAnswer: String,
    @ColumnInfo(defaultValue = "") val explanation: String = "",
    val userAnswer: String,
    val isCorrect: Boolean,
    val confidence: String,                         // "Not sure" | "Maybe" | "I know it"
    @ColumnInfo(defaultValue = "0") val responseTimeMillis: Long = 0L,
    val reviewedAt: Long = System.currentTimeMillis()
)

@Dao
interface ReviewResultDao {
    @Query("SELECT * FROM review_results ORDER BY reviewedAt DESC")
    fun getAllResults(): Flow<List<ReviewResultEntity>>

    @Query("SELECT * FROM review_results WHERE topicId = :topicId ORDER BY reviewedAt
DESC")
    fun getResultsForTopic(topicId: Int): Flow<List<ReviewResultEntity>>

    @Query("SELECT COUNT(*) FROM review_results WHERE topicId = :topicId")
    suspend fun getAnswerCountForTopic(topicId: Int): Int

    @Query("SELECT COUNT(*) FROM review_results WHERE topicId = :topicId AND isCorrect =
1")
    suspend fun getCorrectAnswerCountForTopic(topicId: Int): Int

    @Insert suspend fun insertResult(result: ReviewResultEntity): Long

    @Query("DELETE FROM review_results WHERE topicId = :topicId")
    suspend fun clearTopicResults(topicId: Int)
```

```
}
```

Listing A2. ReviewResultEntity.kt and ReviewResultDao.kt

A.3 Brand colours

```
// Loopi brand palette. Every screen reads from these so the flows stay consistent.
object LoopiColors {
    val Navy = Color(0xFF123B8F)
    val Blue = Color(0xFF2563EB)
    val BlueSoft = Color(0xFFE8F0FE)
    val Teal = Color(0xFF14B89A)
    val Red = Color(0xFFE5484D)
    val Amber = Color(0xFFF5A524)
    val Purple = Color(0xFF7C5CF6)
    val Background = Color(0xFFF6F8FC)
    val Ink = Color(0xFF14213D)
    val Muted = Color(0xFF6B7A99)
    // ... soft variants, Border and Track colours
}
```

Listing A3. ui/theme/Color.kt (abbreviated)

Appendix B. Student Declaration

We declare that we understand what is meant by plagiarism; this assignment is all our own work and we have acknowledged any use of published or unpublished works of other people; and we hold a copy of this assignment which we can produce if the original is lost or damaged.

Name	Student ID	Signature	Date
Akhmetova Azhar	I26043015		
Tussupova Dariya	I26043025		
Zhassyuzak Nursaule	I26043027		
Suzain Mudabbir Sayyad	I26043300		
Sanjeet Sachin Zele	I26043274		